

Eventify: A Microservices-Based Event Management and Ticketing Platform Technical Report

Software Engineering Seminar
Final Project Documentation

Authors:

Carlos Andres Abella
Daniel Felipe Paez
Leidy Marcela Morales

Supervisor:

Eng. Carlos Andrés Sierra, M.Sc.

Universidad Distrital Francisco José de Caldas
Faculty of Engineering
Bogotá, Colombia

December 12, 2025

Contents

Abstract	6
1 Introduction	7
1.1 Background and Context	7
1.2 Problem Statement	7
1.3 Report Objectives	7
1.4 Report Structure	7
2 Literature Review	8
2.1 Microservices in Event and Ticketing Platforms	8
2.2 Distributed Authentication and Security	8
2.3 Operationalization via Containers and CI/CD	8
2.4 Testing Strategies for Distributed Systems	8
3 Background	9
3.1 Technology Stack Overview	9
3.2 Microservices Architecture Principles	9
3.3 Agile Development and Scrum	9
4 Objectives	10
4.1 Primary Objectives	10
4.2 Success Criteria	10
5 Scope	11
5.1 Included in Scope	11
5.2 Excluded from Scope	11
5.3 Project Boundaries	11
6 Assumptions	12
7 Limitations	12
8 Methodology	13
8.1 User Stories and Scrum Implementation	13
8.1.1 User Stories Development	13
8.2 Role: Event Organizer	13
8.3 Role: Ticket Buyer	15
8.4 Role: Platform Admin	17
8.4.1 User Story Mapping	18
8.4.2 Scrum Implementation	21
8.5 System Analysis	28
8.5.1 Actors and Interactions	29
8.6 CRC Cards	30
8.6.1 Class: User	30
8.6.2 Class: PlatformAdmin	30
8.6.3 Class: EventOrganizer	30
8.6.4 Class: Event	31
8.6.5 Class: TicketType	31
8.6.6 Class: Ticket	31

8.6.7	Class: Order	31
8.6.8	Class: Payment	32
8.6.9	Class: TicketBuyer	32
8.6.10	Class: NotificationService	32
8.6.11	Class: Report	32
8.6.12	Class: Review	33
8.6.13	Class: AuditLog	33
8.6.14	Class: Location	33
8.6.15	Class: Category	33
8.7	Class Diagram	33
8.8	Software Architecture Design	35
8.8.1	Architecture Diagram	35
8.8.2	Design Rationale	36
8.9	Implementation Methodology	37
8.9.1	Backend Implementation Approach	37
8.9.2	Frontend Implementation Approach	37
8.9.3	Testing Strategy	37
9	Implementation Details	38
9.1	Backend Implementation	38
9.1.1	Java Authentication Service	38
9.1.2	Python Backend for Business Logic	41
9.1.3	API Endpoints and CRUD Operations	42
9.1.4	Integration and Validation	49
9.2	Frontend Implementation	50
9.2.1	React Application	50
9.2.2	User Interface Evidence	51
10	Testing Implementation	54
10.1	Unit Testing	54
10.1.1	Java Backend (JUnit 5)	54
10.1.2	Python Backend (pytest)	55
10.2	Integration Testing	57
10.3	Acceptance Testing (Cucumber)	57
10.4	Stress Testing (JMeter)	58
10.5	CI/CD and Deployment	58
10.5.1	GitHub Actions CI/CD Pipeline	58
10.6	Docker-Based Containerization and Orchestration	59
11	Results	62
11.1	Unit Testing Results	62
11.1.1	Java Backend Results	62
11.1.2	Python Backend Results	62
11.1.3	Overall Unit Test Summary	63
11.2	Integration Testing Results	63
11.3	Acceptance Testing Results	63
11.4	Stress Testing Results	63
11.5	CI/CD Pipeline Results	64
11.6	Deployment Results	64

12 Discussion	65
12.1 Achievement of Objectives	65
12.2 Architecture Effectiveness	65
12.3 Testing Strategy Effectiveness	65
12.4 CI/CD Pipeline Effectiveness	66
12.5 Containerization Benefits	66
12.6 Limitations and Their Impact	66
12.7 Lessons Learned	66
13 Conclusions	67
13.1 Summary of Achievements	67
13.2 Key Contributions	67
13.3 Future Work Recommendations	67
13.4 Final Remarks	68
Acknowledgements	70
Glossary	71

List of Figures

1	User Story Mapping: Visual representation of the final release	20
2	Sprint Structure:	22
3	Scrum Sprint Cycle	22
4	User Stories Distribution	24
5	Scrum Board: Visual representation of the project's task management	26
6	System diagram	29
7	Class Diagram showing entity relationships, user role hierarchy, and service col- laborations	34
8	Architecture diagram	35
9	Deployment diagram	36
10	MySQL authentication ER diagram	38
11	POST /auth/register request	39
12	POST /auth/register response	40
13	POST /auth/login request	40
14	POST /auth/login response	41
15	Example response from GET /events showing all available events.	42
16	JSON schema used for creating a new event.	43
17	POST /events request and response example in Swagger.	43
18	GET /tickets endpoint retrieving ticket details.	44
19	JSON structure for creating a new ticket type.	44
20	Example POST /tickets request and response.	45
21	Example response from GET /orders with payment information included.	45
22	JSON schema used for creating an order.	46
23	POST /orders request example in Swagger.	46
24	Example response from GET /locations.	47
25	JSON body used to create a new location.	47
26	POST /locations request and response example.	48
27	GET /categories returning available event categories.	48
28	JSON structure for creating a new category.	49
29	POST /categories request and response example in Swagger.	49
30	Authentication flow: login and registration.	51
31	Event exploration and filtering.	51
32	Event detail and ticket selection for purchase.	51
33	Buyer dashboard and order history.	52
34	Notifications center for buyers.	52
35	Organizer dashboard and event management.	52
36	Event creation flow (organizer).	53
37	Admin dashboards with metrics and control panels.	53
38	User management from the admin role.	54
39	JUnit test execution summary showing all tests passed.	55
40	Pytest execution summary showing all tests passed.	56
41	Pylint code quality	57
42	Execution view of the GitHub Actions CI/CD workflow	59
43	Docker Compose deployment	61
44	Docker Desktop	61

List of Tables

1	CRC Card — User Class	30
2	CRC Card — PlatformAdmin Class	30
3	CRC Card — EventOrganizer Class	30
4	CRC Card — Event Class	31
5	CRC Card — TicketType Class	31
6	CRC Card — Ticket Class	31
7	CRC Card — Order Class	31
8	CRC Card — Payment Class	32
9	CRC Card — TicketBuyer Class	32
10	CRC Card — NotificationService Class	32
11	CRC Card — Report Class	32
12	CRC Card — Review Class	33
13	CRC Card — AuditLog Class	33
14	CRC Card — Location Class	33
15	CRC Card — Category Class	33
16	Java Backend Unit Test Results	62
17	Python Backend Unit Test Results	62
18	Overall Unit Test Summary	63
19	Acceptance Test Results	63
20	Stress Test Results Summary	63

Abstract

This technical report presents the conceptual design, systematic implementation, and analytical evaluation of a microservices-oriented platform for event management and ticketing. The work addresses the limitations of monolithic event-management solutions by applying a modular, scalable, and distributed architectural approach. The system is structured around independently deployable services, a decoupled communication model, and a clear separation of concerns, enabling flexibility, maintainability, and horizontal scalability. The report outlines the complete software development lifecycle, including the elicitation and formalization of requirements, iterative planning using agile methodologies, system modeling, and architectural specification grounded in service decomposition and API-driven interaction. It further details the implementation of the platform's core layers, encompassing service orchestration, data management, interface design, and integration logic. Quality assurance is examined through a multi-layered testing strategy that incorporates unit-level validation, behavioral acceptance verification, and performance evaluation under variable load conditions. The deployment process is analyzed through the lens of automated delivery pipelines, emphasizing reproducibility, traceability, and continuous integration principles. The findings indicate effective realization of the platform's functional and architectural objectives, reflected in high verification and validation consistency, stable service interactions, and successful operational deployment. Overall, the project demonstrates the practical application of contemporary software engineering paradigms, including microservices architecture, distributed system design, automated testing frameworks, and continuous software delivery practices. **Keywords:** Microservices Architecture, Event Manage-

ment, Software Engineering, Containerization, RESTful APIs, Testing, CI/CD

1 Introduction

1.1 Background and Context

The event management industry has undergone significant digital transformation over the past decade, driven by the need for efficient ticket distribution, improved customer experience, and streamlined operations. Traditional event management methods suffer from numerous limitations including manual processes leading to revenue loss, inadequate communication channels between organizers and attendees, limited event visibility, and high operational costs. These challenges have motivated the development of digital platforms that automate event lifecycle management from creation to ticket sales and post-event analytics. Modern event management platforms must address several critical requirements: secure user authentication and authorization, real-time ticket availability management, seamless payment processing, comprehensive event discovery mechanisms, and robust notification systems. Additionally, these systems must be scalable to handle peak loads during popular event sales, maintainable to support continuous feature development, and reliable to ensure business continuity.

1.2 Problem Statement

Event organizers need a secure, scalable platform to publish events, sell tickets, and track orders without service degradation during peak sales. The project responds by decomposing the system into independently deployable services, enforcing API-first contracts, and automating deployment and testing to reduce operational risk.

1.3 Report Objectives

This report explains how the platform was conceived, built, and validated: it summarizes the requirements and Scrum cadence that shaped scope; details architecture, APIs, and data models; outlines backend, frontend, and deployment implementation; and presents the evidence from automated tests, performance checks, and containerized runs that confirm the solution meets the stated problem.

1.4 Report Structure

This report is organized into the following sections: Section 2 provides a literature review of relevant research and technologies. Section 3 presents the background and context necessary to understand the project. Section 4 defines the objectives and scope of the project. Section 5 outlines assumptions and limitations. Section 6 describes the methodology, including user stories, Scrum implementation, system analysis, and architecture design. Section 7 presents the implementation details for backend, frontend, testing, and CI/CD. Section 8 presents results from testing and evaluation. Section 9 discusses the findings and their implications. Section 10 concludes with a summary and future work recommendations. Appendices contain supplementary material including API documentation, database schemas, and additional diagrams.

2 Literature Review

2.1 Microservices in Event and Ticketing Platforms

Prior work on ticketing systems highlights scalability and fault isolation as recurring pain points. Richardson [1] and Newman [7] show how decomposing authentication, catalog, and payment domains into services avoids single points of failure and eases independent scaling—patterns we mirror with separate Java (auth) and Python (business) services.

2.2 Distributed Authentication and Security

Studies on secure identity for distributed apps emphasize token-based flows and centralized user stores to protect APIs without coupling services. We adopt JWT-based stateless auth and a dedicated MySQL-bound identity service, aligning with practices reported in industry case studies to reduce cross-service trust assumptions.

2.3 Operationalization via Containers and CI/CD

Turnbull [4] documents container use to keep environments reproducible, while recent DevOps literature stresses pipelines that validate, build, and ship artifacts per change. Our Docker Compose topology and GitHub Actions workflow follow those patterns to ensure consistent deployments across contributors.

2.4 Testing Strategies for Distributed Systems

Research on microservices testing (e.g., contract and resilience testing) notes the need to validate interactions, not just units. We apply a layered approach—unit, integration with real databases, acceptance, and load tests—to mirror the testing pyramid while capturing cross-service behaviors.

3 Background

3.1 Technology Stack Overview

The Eventify platform leverages a modern technology stack designed for scalability, maintainability, and developer productivity:

Frontend: React with TypeScript provides a component-based architecture for building responsive user interfaces. React's virtual DOM enables efficient rendering, while TypeScript adds static type checking for improved code quality.

Backend Services: Java Spring Boot handles authentication and user management, leveraging Spring Security for security features and Spring Data JPA for database access. Python FastAPI manages business logic, providing high performance through asynchronous request handling and automatic API documentation generation.

Data Persistence: MySQL stores authentication-related data, while PostgreSQL manages business domain entities. This separation enables independent scaling and optimization of each database.

DevOps: Docker containerizes all components, ensuring consistent deployment across environments. Docker Compose orchestrates multi-container applications, managing dependencies and networking.

Testing: JUnit 5 tests Java services, pytest tests Python services, Cucumber validates acceptance criteria, and JMeter performs stress testing.

3.2 Microservices Architecture Principles

The microservices architecture pattern provides several key benefits:

- **Technology Diversity:** Each service can use the most appropriate technology stack
- **Independent Deployment:** Services can be updated without affecting others
- **Fault Isolation:** Failures in one service do not cascade to others
- **Scalability:** Services can scale independently based on load patterns

3.3 Agile Development and Scrum

The project follows Agile development principles, specifically implementing Scrum methodology. Scrum emphasizes iterative development, regular feedback, and adaptive planning. Key Scrum practices include sprint planning, daily stand-ups, sprint reviews, and retrospectives.

4 Objectives

4.1 Primary Objectives

The project objectives are framed as business outcomes and supporting enablers:

- Deliver a reliable event ticketing capability (publish events, sell tickets, track orders) that stays responsive under peak load.
- Secure user identity and access with a dedicated authentication service and audited data store.
- Validate functionality and quality through automated unit, integration, acceptance, and load tests, reaching high coverage and zero regression at release.
- Ensure repeatable delivery via CI/CD and containerization so the full stack can be deployed locally or in the cloud with minimal effort.
- Provide a usable web experience for organizers and buyers that integrates both backends through stable REST APIs.

4.2 Success Criteria

The project will be considered successful if:

- All user stories are implemented and validated through acceptance tests
- Test coverage exceeds 75% for both backend services
- All acceptance test scenarios pass
- CI/CD pipeline successfully builds and tests the application
- Application runs successfully in Docker containers
- Documentation is complete and comprehensive

5 Scope

5.1 Included in Scope

This project includes:

- **User Stories and Scrum Implementation:** Comprehensive user stories for all user roles, user story mapping, and detailed Scrum implementation report.
- **System Analysis:** Complete system analysis identifying components, actors, and interactions.
- **Architecture Design:** Software architecture design using object-oriented principles, including diagrams and design rationale.
- **Backend Implementation:** Java authentication service (MySQL) and Python business logic service (PostgreSQL) with REST API endpoints.
- **Frontend Implementation:** React-based web interface interacting with both backend services.
- **Testing:** Unit tests (JUnit, pytest), acceptance tests (Cucumber), and stress tests (JMeter).
- **CI/CD:** GitHub Actions workflow for automated testing and Docker image building.
- **Containerization:** Docker and Docker Compose configuration for all components.
- **Documentation:** Comprehensive documentation covering all project aspects.

5.2 Excluded from Scope

This project intentionally omits a set of functionalities and architectural components that, while relevant in production-grade systems, fall outside the boundaries defined for this academic implementation. These exclusions include deployment to cloud platforms, integration with real payment gateways, development of a dedicated mobile application, and the incorporation of advanced monitoring, observability, and caching mechanisms. Additionally, the system does not include email service integration beyond simulated notifications, nor does it provide multi-language support or advanced analytics and reporting capabilities. These elements are excluded to maintain a feasible scope aligned with the project's objectives and constraints.

5.3 Project Boundaries

The project focuses on demonstrating software engineering practices rather than building a production-ready commercial platform. The scope emphasizes learning and applying modern software development methodologies, architectural patterns, and testing strategies within an academic context.

6 Assumptions

The following assumptions were made during the project:

1. **Development Environment:** The project assumes developers have access to modern development tools including Java 17+, Python 3.12+, Node.js 16+, Docker, and Git.
2. **Database Availability:** MySQL and PostgreSQL databases are available either locally or through Docker containers.
3. **Network Configuration:** All services communicate over HTTP/HTTPS protocols, and CORS is properly configured for frontend-backend communication.
4. **User Trust:** Event organizers are assumed to be trusted entities with limited fraud prevention mechanisms beyond basic validation.
5. **Single Deployment Environment:** The system assumes a single deployment environment without geographic distribution considerations.
6. **Payment Processing:** Payment processing is simulated rather than integrated with real payment gateways, avoiding PCI DSS compliance requirements.
7. **Event Organizer Verification:** Basic validation is assumed sufficient, including email format verification and password security requirements.
8. **Database Transactions:** Transactions are managed at the service level without distributed transaction management across services.
9. **Caching:** The system does not implement advanced caching strategies, relying on direct database queries.
10. **Session Management:** Stateless authentication using JWT tokens eliminates the need for server-side session storage.

7 Limitations

This project is constrained by several factors that influence both the scope and the evaluative depth of the outcomes. The academic timeframe restricts the level of completeness achievable, limiting the incorporation of advanced functionalities and extensive experimentation. Additionally, the absence of production-grade infrastructure reduces the capacity to perform evaluations under realistic operational loads, particularly those involving global scalability, latency distribution, and high-demand scenarios. The implementation relies on simulated payment mechanisms, which prevents full validation of real-world transactional behaviors and associated security requirements, while the lack of robust fraud-prevention capabilities further limits the system's readiness for deployment in high-risk environments. Architectural restrictions are also present; the system does not incorporate distributed transaction management, making data consistency across services reliant on application-level coordination, and it excludes dedicated caching layers, which may affect performance under intensive workloads. Operational visibility is similarly constrained by limited monitoring and observability tooling, reducing the ability to perform granular performance analysis or real-time issue detection. Finally, despite the adoption of a comprehensive testing strategy, test coverage remains incomplete, leaving a subset of edge-case behaviors unexplored. Collectively, these limitations contextualize the project's results and delineate opportunities for targeted enhancement in future work.

8 Methodology

This section describes the methods and procedures used throughout the project, including user story development, Scrum implementation, system analysis, architecture design, and implementation approaches.

8.1 User Stories and Scrum Implementation

8.1.1 User Stories Development

User stories constitute a fundamental agile mechanism for understanding and documenting system functionalities from the perspective of the actors who interact with the platform. Formulated in a concise structure—*As a [role], I want to [action], so that [benefit]*—they capture the specific needs, objectives, and expected value for each user type, ensuring that development remains aligned with real user requirements. This approach enables a clear representation of the priorities and motivations of buyers, sellers, administrators, and other roles within the system. Each story is complemented by well-defined acceptance criteria that establish the conditions under which a feature is considered complete and valid, thereby facilitating verification and guiding implementation. Through this user-centered methodology, the project ensures that the platform evolves toward intuitive, purposeful functionality that effectively addresses the problems and expectations of its various stakeholders.

8.2 Role: Event Organizer

Title: Create and Publish a New Event	Priority: High	Estimate: 8 weeks
--	-----------------------	--------------------------

User Story:
As an **Event Organizer**, I want to **create and publish a new event with detailed information, seating types, and prices**, so that **I can make it available for ticket buyers on the platform**.

Acceptance Criteria:
Given I am logged in as an "Event Organizer",
When I navigate to the "Create Event" page, fill in all mandatory fields, define at least two ticket tiers, and click the "Publish" button,
Then the event is assigned a unique URL, becomes publicly visible, and its tickets are available for purchase.

Title: View Real-Time Sales Dashboard	Priority: High	Estimate: 4 weeks
--	-----------------------	--------------------------

User Story:

As an **Event Organizer**, I want to **view a real-time sales dashboard for my event**, so that **I can track ticket sales and revenue**.

Acceptance Criteria:

Given I am logged in as an "Event Organizer" and am viewing one of my active events,

When I click on the "Dashboard" or "Reports" tab,

Then the system displays the total number of tickets sold per tier and the total gross revenue, updated in near real-time.

Title: Register and Login to Platform	Priority: High	Estimate: 3 weeks
--	-----------------------	--------------------------

User Story:

As an **Event Organizer**, I want to **register an account and log in to the platform**, so that **I can access the system and create events**.

Acceptance Criteria:

Given I am a new user on the platform,

When I provide a valid email and password to register, and then use those credentials to log in,

Then my account is created, I receive a JWT token, and I can access the organizer dashboard.

Title: Manage Event Details	Priority: Medium	Estimate: 4 weeks
------------------------------------	-------------------------	--------------------------

User Story:

As an **Event Organizer**, I want to **update event information and manage ticket availability**, so that **I can keep event details current and control ticket sales**.

Acceptance Criteria:

Given I am logged in as an "Event Organizer" and have created an event,

When I navigate to the event management page and modify event details, ticket prices, or availability,

Then the changes are saved and reflected immediately in the event listing.

8.3 Role: Ticket Buyer

Title: Purchase Event Ticket Securely	Priority: High	Estimate: 8 weeks
--	-----------------------	--------------------------

User Story:

As a **Ticket Buyer**, I want to **select tickets for an event and complete the purchase securely**, so that **I can receive a valid digital ticket to attend**.

Acceptance Criteria:

Given I am viewing the page for an event with available tickets,

When I select my tickets, proceed to checkout, and confirm the mock payment,

Then I am redirected to a confirmation page with the digital tickets as a PDF with a unique QR code.

Title: Browse and Discover Events	Priority: High	Estimate: 5 weeks
--	-----------------------	--------------------------

User Story:

As a **Ticket Buyer**, I want to **search for events by name, date, or category**, so that **I can easily find events I'm interested in**.

Acceptance Criteria:

Given I am on the homepage,

When I use the search bar or apply a category filter,

Then the search results page displays only the events that match my criteria.

Title: Register and Login to Platform	Priority: High	Estimate: 3 weeks
--	-----------------------	--------------------------

User Story:

As a **Ticket Buyer**, I want to **register an account and log in to the platform**, so that **I can purchase tickets and manage my orders**.

Acceptance Criteria:

Given I am a new user on the platform,

When I provide a valid email and password to register, and then use those credentials to log in,

Then my account is created, I receive a JWT token, and I can browse and purchase tickets.

Title: View Order History	Priority: Medium	Estimate: 3 weeks
----------------------------------	-------------------------	--------------------------

User Story:

As a **Ticket Buyer**, I want to **view my order history**, so that **I can track my past purchases and access my tickets**.

Acceptance Criteria:

Given I am logged in as a "Ticket Buyer" and have made purchases,

When I navigate to the "My Orders" section,

Then I can see a list of all my orders with event details, purchase dates, ticket quantities, and order status.

Title: View Event Details	Priority: High	Estimate: 2 weeks
----------------------------------	-----------------------	--------------------------

User Story:

As a **Ticket Buyer**, I want to **view detailed information about an event**, so that **I can make informed decisions before purchasing tickets**.

Acceptance Criteria:

Given I am browsing available events,

When I click on an event to view its details,

Then I can see complete information including event description, date, location, available ticket types, prices, and remaining availability.

8.4 Role: Platform Admin

Title: View Platform Sales Dashboard	Priority: Medium	Estimate: 4 weeks
---	-------------------------	--------------------------

User Story:

As a **Platform Admin**, I want to **view a dashboard with platform-wide sales statistics and revenue data**, so that **I can monitor overall system health and business performance**.

Acceptance Criteria:

Given I am logged in as a "Platform Admin" and navigate to the "Dashboard" section,

When I access the platform sales dashboard,

Then the system displays aggregated statistics including total sales, revenue by period, number of active events, and overall platform performance metrics, updated in near real-time.

Title: Manage User Accounts	Priority: Medium	Estimate: 4 weeks
User Story: As a Platform Admin , I want to view and manage user accounts , so that I can maintain platform integrity and assist users .		
Acceptance Criteria: Given I am logged in as a "Platform Admin", When I navigate to the user management section, Then I can view a list of all users, filter by role, view user details, and perform account management actions.		

Title: Monitor Platform Events	Priority: Low	Estimate: 3 weeks
User Story: As a Platform Admin , I want to view and monitor all events on the platform , so that I can ensure content quality and compliance .		
Acceptance Criteria: Given I am logged in as a "Platform Admin", When I navigate to the events management section, Then I can see all events on the platform, filter by status, view event details, and take moderation actions if needed.		

8.4.1 User Story Mapping

User Story Mapping is a software engineering technique used to visualize, organize, and prioritize product functionalities from the user's perspective. Its main goal is to understand how different user roles interact with the system and what steps they follow to achieve their objectives, dividing this journey into *epics (big activities)*, *features (specific actions)*, and *releases (incremental versions)* of the product.

In our project, which consists of an event ticketing and management platform, we used this approach to represent the entire system flow from event creation to ticket purchase and monitoring.

We identified three main user personas:

- **Event Organizer:** creates and publishes events, defines ticket prices, and tracks sales.
- **Ticket Buyer:** searches for events, selects, and purchases tickets securely.
- **Platform Admin:** supervises the platform, manages accounts, and prevents fraud.

From these personas, we defined the following main epics:

1. **Account Management:** sign up, login, and profile management.
2. **Event Creation & Publishing:** create events, define ticket tiers, and publish them.
3. **Event Discovery:** search and filter available events.
4. **Ticket Selection & Purchase:** select ticket type and quantity, checkout, and payment.
5. **Monitoring & Administration:** sales dashboard and account control.

Each epic was broken down into **features** with detailed functionalities, and the user stories identified for each role were mapped to these features. For example, under "Ticket Purchase" we included features such as *Select Tickets*, *Checkout*, and *Payment*, which correspond to the user story "Purchase Event Ticket Securely" for Ticket Buyers. Similarly, the "Event Creation & Publishing" epic encompasses features like *Create Event*, *Define Ticket Tiers*, and *Publish Event*, mapped to the "Create and Publish a New Event" user story for Event Organizers.

The user stories were distributed across **four main releases**, each corresponding to a two-week delivery period aligned with the course workshop deadlines. These versions were structured as follows:

1. **Version 1:** Focused on requirements elicitation, user stories definition, and system modeling. Included user stories: Register and Login (Organizer and Buyer), Browse and Discover Events (Buyer).
2. **Version 2:** Emphasized architecture design and detailed system specifications. Included user stories: Create and Publish Event (Organizer), View Event Details (Buyer), Manage User Accounts (Admin).
3. **Version 3:** Covered backend and frontend implementation with core functionalities. Included user stories: Purchase Event Ticket (Buyer), View Real-Time Sales Dashboard (Organizer), Manage Event Details (Organizer), View Order History (Buyer).
4. **Version 4:** Focused on testing, containerization, and CI/CD implementation. Included user stories: View Event Orders (Organizer), View Platform Sales Dashboard (Admin), Monitor Platform Events (Admin).

The result was a clear and organized **visual User Story Map** showing how the product evolves, which features are prioritized, and how each user gains value throughout the process. This mapping helped us plan development iterations efficiently according to the course schedule, maintain a user-centered vision of the system's evolution, and ensure that each two-week release delivered tangible value to the different user personas.

The User Story Map was organized into **four separate versions**, one for each release, allowing clear visualization of the incremental delivery of functionality. Each version represents the complete map of features and user stories for that specific release, showing the epics, features, and their relationships within that iteration.

User Story Mapping

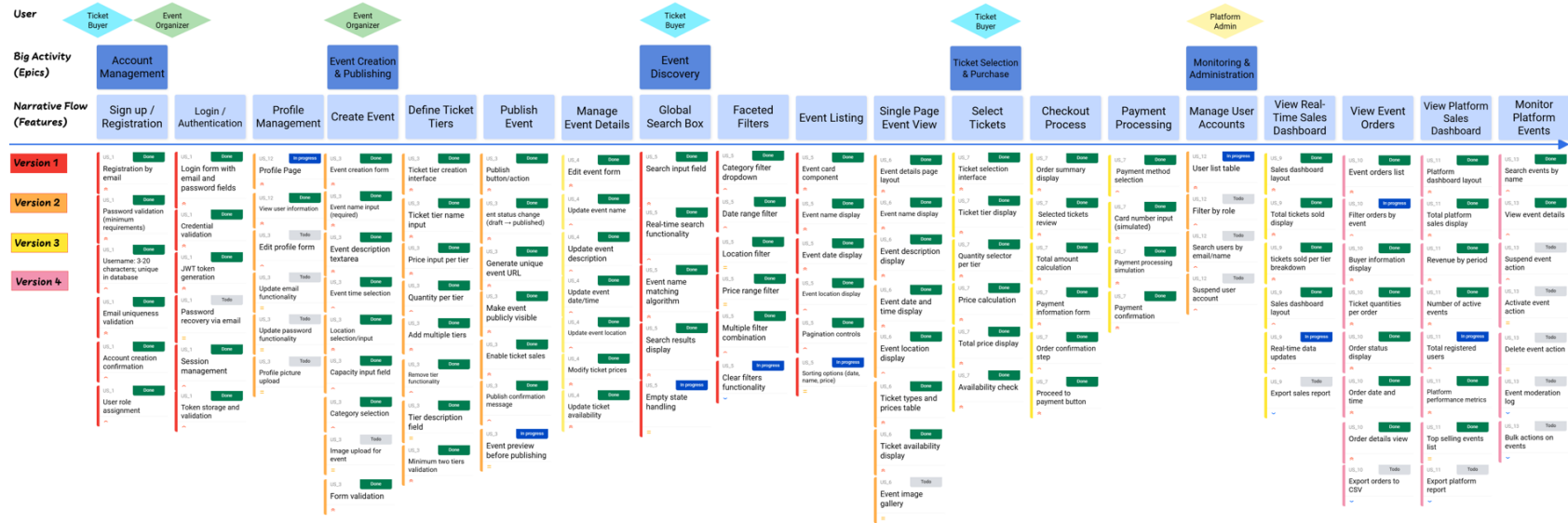


Figure 1: User Story Mapping: Visual representation of the final release

Figure 1 shows the final version of the User Story Mapping.

8.4.2 Scrum Implementation

The Scrum framework was adopted as the primary agile methodology to guide the project's iterative development, ensuring continuous alignment between the system's evolving functionalities and the user-centered requirements captured through User Stories and the User Story Mapping. Given that the project defined three main personas (Event Organizer, Ticket Buyer, and Platform Admin) and decomposed their needs into epics, features, and detailed user stories, Scrum provided a structured mechanism to progressively deliver value across the four planned releases.

To operationalize this approach, the team conducted **Sprint Planning** at the beginning of each iteration, using the User Story Map as the main input for prioritization. Each sprint defined a set of stories aligned with the corresponding release scope (Workshop 1–4), ensuring that requirements flowed coherently from elicitation to modeling, implementation, and deployment. During planning, stories were estimated using relative effort, broken down into technical tasks for each microservice, and assigned according to team capacity. This ensured traceability between high-level features (e.g., “Create Event”, “Ticket Purchase”, “Sales Dashboard”) and the implementation tasks required to deliver them.

Although Scrum formally prescribes a Daily Scrum every 24 hours, the team adapted this cadence to the academic setting by conducting **weekly coordination meetings**. Despite the reduced frequency, these meetings fulfilled the intended purpose of progress inspection, early impediment detection, task synchronization, and short-term planning, ensuring that each sprint advanced toward its goal.

Throughout development, **continuous backlog refinement** was performed to update priorities based on the evolving understanding of user needs, technical constraints, and the dependencies revealed during implementation of the microservices architecture. This refinement ensured that stories remained well-defined, feasible within the sprint, and aligned with the epics defined in the User Story Map—such as Account Management, Event Creation and Publishing, Event Discovery, Ticket Purchase, and Platform Monitoring.

At the end of each sprint, a **Sprint Review** was held to demonstrate completed functionalities mapped to the release objectives. Examples included user authentication (Release 1), event creation flows and admin controls (Release 2), ticket purchase and dashboards (Release 3), and containerization, testing, and CI/CD pipelines (Release 4). These review meetings validated the acceptance criteria of each story and incorporated feedback from stakeholders.

Each sprint also concluded with a **Sprint Retrospective**, where the team assessed collaboration practices, tooling reliability, division of responsibilities, and integration workflows across services. The insights from these retrospectives informed process improvements for the next iteration, such as refining branching strategies, adjusting microservice API contracts, and reorganizing tasks to reduce bottlenecks.

Through this structured yet flexible application of Scrum principles, the project maintained a coherent flow from user-centered requirements to implemented functionality. By integrating the User Story Map into sprint planning and release management, the methodology ensured that development was incremental, testable, traceable, and closely aligned with the evolving expectations of the Event Organizer, Ticket Buyer, and Platform Admin roles. As a result, Scrum served as a unifying framework that coordinated requirement definition, architectural decisions, implementation activities, validation processes, and deployment practices throughout the entire duration of the project.

Sprint Structure The project was organized into four two-week sprints, each corresponding to a course workshop delivery. This structure ensured incremental delivery of value while meeting academic milestones.

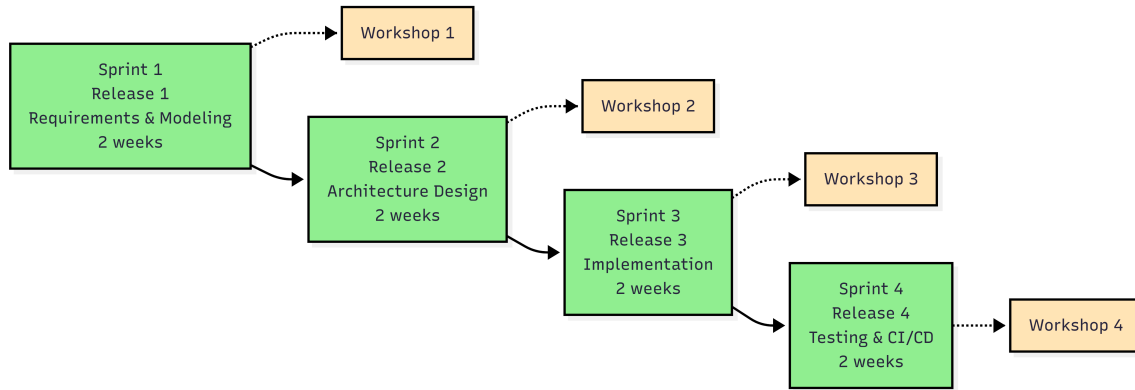


Figure 2: Sprint Structure:

Figure 2 illustrates the four-sprint structure. Each sprint had a specific focus aligned with its corresponding workshop: Sprint 1 focused on requirements and modeling, Sprint 2 on architecture design, Sprint 3 on implementation, and Sprint 4 on testing and CI/CD.

Scrum Events and Practices The team implemented the core Scrum events, adapted to the academic context and project requirements.

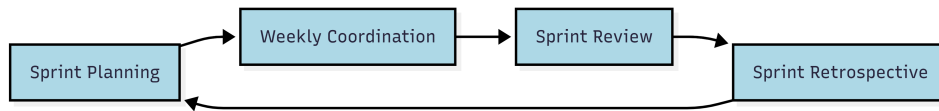


Figure 3: Scrum Sprint Cycle

Figure 3 illustrates the Scrum sprint cycle implemented throughout the project. The team followed this iterative process for each of the four two-week sprints, with adaptations to fit the academic context.

Sprint Planning:

At the beginning of each iteration, the team conducted sprint planning sessions using the User Story Map as the main input for prioritization. Each sprint defined a set of stories aligned with the corresponding release scope (Workshop 1–4), ensuring that requirements flowed coherently from elicitation to modeling, implementation, and deployment. The planning process included:

- Selecting user stories from the User Story Map based on release priorities
- Estimating stories using relative effort (story points)
- Breaking down stories into technical tasks for each microservice (Java backend, Python backend, React frontend)
- Defining sprint goals aligned with release objectives
- Identifying dependencies between tasks and prioritizing blocking items

This process ensured traceability between high-level features (e.g., "Create Event", "Ticket Purchase", "Sales Dashboard") and the implementation tasks required to deliver them.

Weekly Coordination Meetings:

Although Scrum formally prescribes a Daily Scrum every 24 hours, the team adapted this cadence to the academic setting by conducting weekly coordination meetings. Despite the reduced frequency, these meetings fulfilled the intended purpose of:

- Progress inspection and status updates
- Early impediment detection and resolution
- Task synchronization across team members
- Short-term planning adjustments

These meetings were particularly critical for coordinating work across multiple services (Java authentication backend, Python business logic backend, and React frontend), preventing integration issues and ensuring alignment.

Continuous Backlog Refinement:

Throughout development, continuous backlog refinement was performed to update priorities based on the evolving understanding of user needs, technical constraints, and the dependencies revealed during implementation of the microservices architecture. This refinement ensured that stories remained well-defined, feasible within the sprint, and aligned with the epics defined in the User Story Map—such as Account Management, Event Creation & Publishing, Event Discovery, Ticket Purchase, and Platform Monitoring.

Sprint Reviews:

At the end of each sprint, a Sprint Review was held to demonstrate completed functionalities mapped to the release objectives. These reviews served multiple purposes beyond simple demonstration: they were opportunities to receive and incorporate feedback from course instructors and stakeholders, which directly informed improvements in subsequent iterations.

Examples of deliverables presented in each sprint review included:

- **Sprint 1:** User authentication flows, event discovery interfaces, and system modeling artifacts (class diagrams, use case diagrams)
- **Sprint 2:** Architecture diagrams, event creation flows, and admin control interfaces
- **Sprint 3:** Ticket purchase workflows, sales dashboards, and order management features
- **Sprint 4:** Containerization setup, testing frameworks, and CI/CD pipelines

Beyond validating acceptance criteria, these review meetings incorporated critical feedback from workshop evaluations that led to significant improvements throughout the project. Key feedback-driven enhancements included:

- **Design Improvements:** Refinements to class diagram definitions based on instructor feedback, ensuring better representation of domain entities and their relationships
- **Architecture Corrections:** Adjustments to the microservices architecture based on review feedback, including clearer service boundaries and improved communication patterns
- **User Story Refinement:** Evolution of user stories based on feedback, resulting in more precise acceptance criteria and better alignment with user needs

- **User Story Mapping Optimization:** Restructuring of the User Story Mapping to improve version separation, ensuring clearer distribution of features across the four releases and better traceability between epics, features, and user stories

This iterative feedback loop ensured that the project not only met the acceptance criteria of each story but also continuously improved its design quality, architectural coherence, and requirement clarity based on expert evaluation. The integration of workshop feedback into sprint reviews transformed these events from simple demonstrations into active learning and improvement opportunities, ensuring alignment with course requirements, academic standards, and user expectations.

Sprint Retrospectives:

Each sprint concluded with a Sprint Retrospective, where the team assessed:

- Collaboration practices and team dynamics
- Tooling reliability and development environment effectiveness
- Division of responsibilities and workload distribution
- Integration workflows across microservices
- Process efficiency and bottlenecks

The insights from these retrospectives informed process improvements for the next iteration, such as refining branching strategies, adjusting microservice API contracts, reorganizing tasks to reduce bottlenecks, and improving communication protocols.

Integration with User Story Mapping The User Story Map served as the product backlog, with epics and features guiding sprint planning. Stories were prioritized based on dependencies between features (e.g., authentication must precede event creation), value delivery to different user personas, technical feasibility, and workshop requirements.

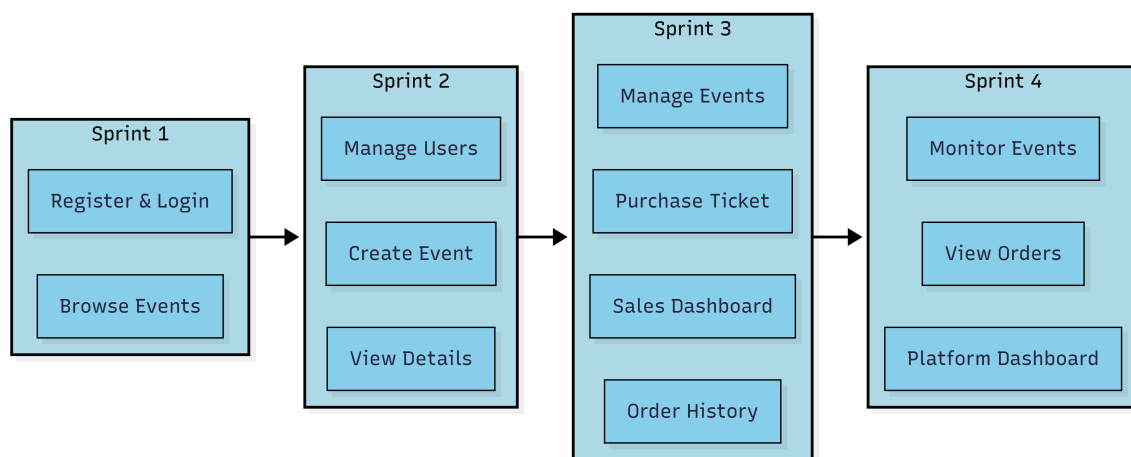


Figure 4: User Stories Distribution

Figure 4 illustrates how user stories were distributed across sprints, ensuring incremental value delivery. By integrating the User Story Map into sprint planning and release management, the methodology ensured that development was incremental, testable, traceable, and closely aligned with the evolving expectations of the Event Organizer, Ticket Buyer, and Platform Admin roles.

Scrum Board and Task Management To track the progress of user stories and their associated tasks throughout the sprints, the team maintained a Scrum Board that visualized the workflow from backlog to completion. The board was organized into four main columns: **Backlog** (items not yet started), **To do** (tasks ready to be worked on), **In progress** (tasks currently being developed), and **Done** (completed tasks). Each user story was represented as a row, with its associated activities distributed across these columns based on their current status.

Scrum Board - Eventify Platform

Deliver a complete event ticketing platform with user authentication, event management, ticket purchase, and administrative dashboards across four releases aligned with course workshops.

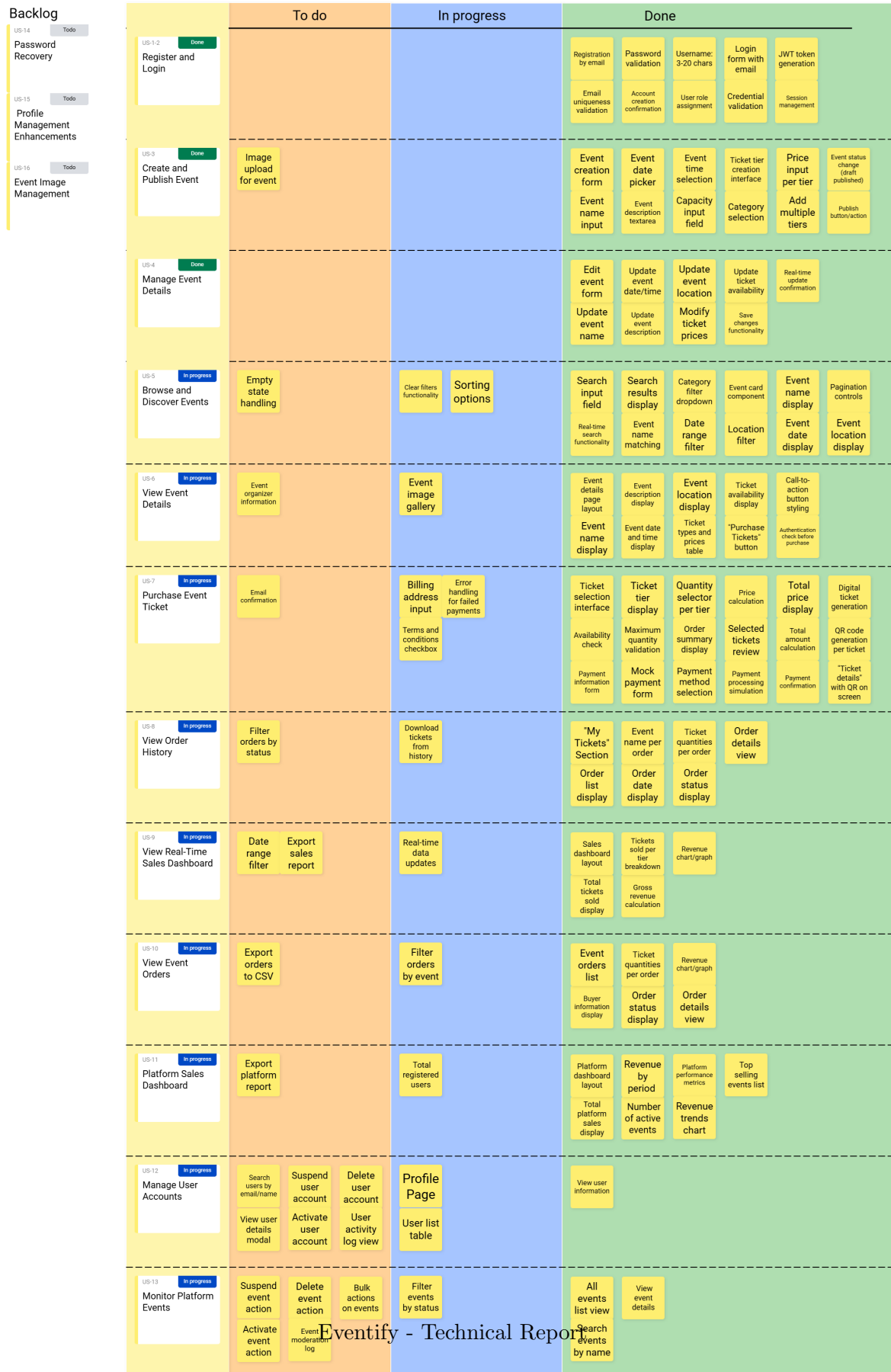


Figure 5: Scrum Board: Visual representation of the project's task management

Figure 5 shows the Scrum Board as it evolved throughout the project. The board provided real-time visibility into:

- **Progress Tracking:** Each user story's completion status, with activities clearly categorized by their current state
- **Workflow Management:** The flow of tasks from backlog through to completion, enabling the team to identify bottlenecks and prioritize work
- **Transparency:** Clear visualization of what was completed, what was in progress, and what remained to be done
- **Sprint Planning Support:** The board served as a reference during sprint planning to identify which stories were ready to be moved into the active sprint

The board structure aligned with the User Story Mapping, ensuring that each epic and feature had corresponding tasks tracked on the board. This integration between the User Story Map (product backlog) and the Scrum Board (sprint backlog) enabled the team to maintain traceability from high-level requirements to specific implementation tasks, facilitating effective sprint planning and progress monitoring throughout all four releases.

Key Learnings and Outcomes Through this structured yet flexible application of Scrum principles, the project maintained a coherent flow from user-centered requirements to implemented functionality. Key learnings from the Scrum implementation include:

- **User Story Mapping as Backlog:** Using the User Story Map as the product backlog provided clear visibility into feature priorities and user value delivery across releases.
- **Release-Based Sprints:** Aligning sprints with course workshop deadlines ensured that each sprint delivered tangible, demonstrable value while meeting academic requirements.
- **Cross-Service Coordination:** Weekly coordination meetings were essential for coordinating work across the three different technology stacks (Java, Python, React), preventing integration issues.
- **Incremental Value Delivery:** Each sprint delivered working functionality that could be demonstrated, from initial modeling (Sprint 1) to fully functional system with testing (Sprint 4).
- **Continuous Improvement:** Sprint retrospectives enabled the team to adapt processes, improving estimation accuracy and task breakdown over time.

Challenges and Solutions The team encountered several challenges during Scrum implementation, which were addressed through adaptive solutions:

- **Challenge:** Coordinating work across multiple services (Java Spring Boot, Python FastAPI, React) with different technology stacks.
- **Solution:** Defined clear API contracts early in Sprint 2, used Swagger/OpenAPI documentation for interface specification, and conducted weekly coordination meetings to synchronize integration efforts.
- **Challenge:** Managing dependencies between team members' tasks, especially when back-end services needed to be ready before frontend integration.

- **Solution:** Used dependency mapping in sprint planning, prioritized blocking tasks (e.g., authentication service before business logic), and implemented parallel work streams where possible (e.g., database design while API contracts were being defined).
- **Challenge:** Balancing feature development with testing and documentation within tight two-week sprint cycles.
- **Solution:** Allocated dedicated time for testing and documentation in each sprint, integrated testing as part of the development process (TDD where applicable), and used automated testing to reduce manual verification time.
- **Challenge:** Ensuring all user stories were properly implemented and validated within the academic timeline.
- **Solution:** Used acceptance criteria from user stories to guide development and testing, implemented Cucumber acceptance tests to validate user story completion, and maintained traceability between user stories and implemented features.

By integrating the User Story Map into sprint planning and release management, Scrum served as a unifying framework that coordinated requirement definition, architectural decisions, implementation activities, validation processes, and deployment practices throughout the entire duration of the project.

8.5 System Analysis

An event management system can be divided into several parts, or subsystems, that work together to meet the overall objectives of successful event planning, management, and execution. The main parts are:

- **Event Management Subsystem:** Includes event creation, editing, publishing, and management. Details such as dates, locations, ticket types, capacity, and logistics are defined here.
- **Registration and Sales Subsystem:** Handles participant registration, ticket sales, and secure payment processing, ensuring digital ticket issuance and validation.
- **Communication and Notifications Subsystem:** Manages contact with organizers and attendees through emails, SMS messages, alerts, and real-time updates.
- **Monitoring and Reporting Subsystem:** Provides dashboards and reports for real-time tracking of sales, capacity, revenue, and post-event feedback for analysis and continuous improvement.
- **Integration Subsystem:** Allows the platform to connect with external services such as payment gateways, CRM systems, social networks, and other digital tools to expand functionality.

Each subsystem is an autonomous yet interrelated component, forming a complex system that interacts with its environment through inputs (user data, requests, payments) and outputs (tickets, notifications, reports). This division facilitates modular management, scalability, and resilience to changes or failures.

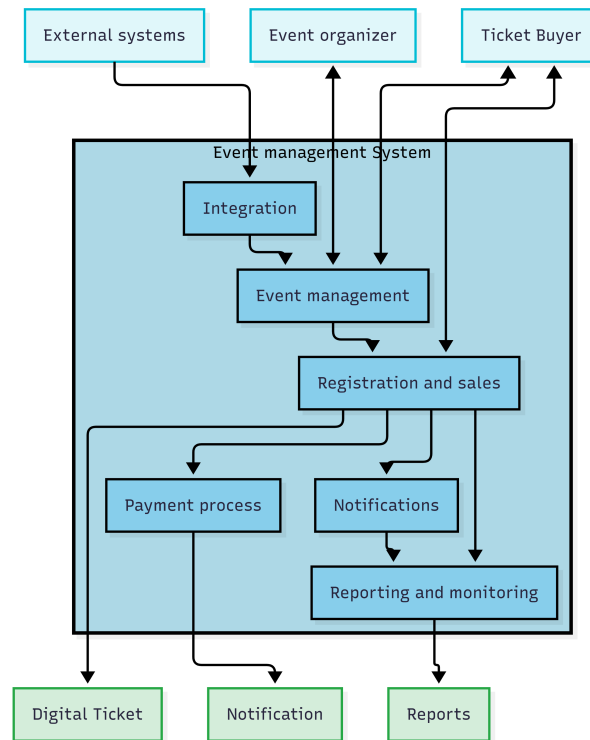


Figure 6: System diagram

This structure allows addressing critical points in the system, especially during high-demand scenarios such as rapid ticket sales (flash sales). In these cases, a small error in the service fee or an incorrect configuration of event capacity can quickly escalate: a flash sale with poorly set capacity can lead to overbooking, multiple claims, and system saturation, seriously affecting the platform's reputation and financial stability.

Furthermore, the architecture must guarantee payment security, the integrity of event and user data, and operational efficiency at all times. These conditions reflect the chaotic and dynamic nature of the real-world environment, where small failures or incorrect decisions can have significant consequences if not properly controlled through validation, monitoring, and automated recovery mechanisms.

8.5.1 Actors and Interactions

The system involves three main actors:

1. **Event Organizer:** Creates and manages events, views dashboard statistics, manages ticket availability.
2. **Ticket Buyer:** Browses events, purchases tickets, views order history, receives digital tickets.
3. **Platform Administrator:** Manages user accounts, monitors platform statistics, handles fraud prevention.

Interactions between actors and the system follow RESTful API patterns, with authentication handled through JWT tokens.

8.6 CRC Cards

CRC (Class–Responsibility–Collaborator) cards provide an object-oriented foundation for the platform’s internal design. Each card identifies a core class, outlining responsibilities and collaborations with other system entities, establishing the link between requirements and class implementation.

The domain classes span core entities (Event, Ticket, Order, Payment, Location) and user roles (EventOrganizer, TicketBuyer, PlatformAdmin), each defining how responsibilities are distributed and how services interact to enable event publishing, sales, and user communication.

8.6.1 Class: User

Class: User	
Responsibility	Collaborator
• Store user profile information (name, email, password, role). • Authenticate and authorize system access. • Manage user sessions and credentials. • Provide common behavior for specialized user types.	• PlatformAdmin • EventOrganizer • TicketBuyer

Table 1: CRC Card — User Class

8.6.2 Class: PlatformAdmin

Class: PlatformAdmin	
Responsibility	Collaborator
• Manage system configuration and global settings. • Monitor platform performance and security. • Access and analyze audit logs. • Oversee user accounts and permissions.	• AuditLog • User

Table 2: CRC Card — PlatformAdmin Class

8.6.3 Class: EventOrganizer

Class: EventOrganizer	
Responsibility	Collaborator
• Create and publish events. • Manage event details, ticket types, and availability. • Monitor sales and generate event reports. • Interact with buyers and respond to feedback.	• Event • Report • TicketBuyer

Table 3: CRC Card — EventOrganizer Class

8.6.4 Class: Event

Class: Event	
Responsibility	Collaborator
• Maintain event information (title, date, location, description). • Define and manage ticket categories (VIP, General, etc.). • Track total capacity, sold tickets, and availability in real time. • Handle event lifecycle: Draft, Published, Sold Out, Cancelled. • Provide performance data to generate reports. • Notify organizers and buyers about event updates.	• EventOrganizer • TicketType • Location • NotificationService • Report • Review • Category

Table 4: CRC Card — Event Class

8.6.5 Class: TicketType

Class: TicketType	
Responsibility	Collaborator
• Define ticket categories (VIP, General, Early Bird, etc.). • Set price, availability, and purchase limits. • Generate corresponding tickets upon purchase. • Update availability based on sales.	• Event • Ticket

Table 5: CRC Card — TicketType Class

8.6.6 Class: Ticket

Class: Ticket	
Responsibility	Collaborator
• Store ticket information (QR code, seat, status). • Link each ticket to its type, order, and buyer. • Validate entry at the event venue. • Update status when used or refunded.	• TicketType • Order • TicketBuyer

Table 6: CRC Card — Ticket Class

8.6.7 Class: Order

Class: Order	
Responsibility	Collaborator
• Record ticket purchases by buyers. • Manage order details: items, total amount, and status. • Process and confirm payments. • Link generated tickets and payment transactions. • Notify buyer upon successful purchase or failure.	• TicketBuyer • Ticket • Payment • NotificationService

Table 7: CRC Card — Order Class

8.6.8 Class: Payment

Class: Payment	
Responsibility	Collaborator
• Process and validate payment transactions. • Support multiple payment methods (card, transfer, etc.). • Track payment status (Pending, Approved, Rejected). • Generate transaction receipts.	• Order

Table 8: CRC Card — Payment Class

8.6.9 Class: TicketBuyer

Class: TicketBuyer	
Responsibility	Collaborator
• Browse and select events to attend. • Purchase tickets and manage orders. • Receive notifications about purchases or event updates. • Provide reviews after attending events.	• Event • Order • NotificationService • Review

Table 9: CRC Card — TicketBuyer Class

8.6.10 Class: NotificationService

Class: NotificationService	
Responsibility	Collaborator
• Send notifications to users about events, purchases, or updates. • Manage notification templates and delivery channels (email, SMS). • Log notification history for tracking.	• User • Notification

Table 10: CRC Card — NotificationService Class

8.6.11 Class: Report

Class: Report	
Responsibility	Collaborator
• Compile event and sales statistics. • Generate analytical summaries for organizers and admins. • Support export in various formats (PDF, Excel, etc.).	• Event • EventOrganizer

Table 11: CRC Card — Report Class

8.6.12 Class: Review

Class: Review	
Responsibility	Collaborator
• Store user feedback and ratings for events. • Link reviews with corresponding buyers and events. • Provide input for quality analysis and reports.	• Event • TicketBuyer

Table 12: CRC Card — Review Class

8.6.13 Class: AuditLog

Class: AuditLog	
Responsibility	Collaborator
• Record administrative actions and system events. • Provide traceability for platform operations. • Support compliance and security reviews.	• PlatformAdmin

Table 13: CRC Card — AuditLog Class

8.6.14 Class: Location

Class: Location	
Responsibility	Collaborator
• Store and manage event venue data (address, capacity). • Provide location details for event display and logistics. • Support updates for address or capacity changes.	• Event

Table 14: CRC Card — Location Class

8.6.15 Class: Category

Class: Category	
Responsibility	Collaborator
• Classify events by category (Music, Sports, Theater, etc.). • Support browsing and filtering by category. • Help generate category-based reports.	• Event

Table 15: CRC Card — Category Class

8.7 Class Diagram

The Class Diagram translates CRC cards into a structured blueprint showing classes, attributes, methods, and relationships. It applies object-oriented principles—encapsulation, inheritance, and composition—to represent how user roles, business entities, and services interact.

Key structural elements include a User hierarchy extended by specialized subclasses (EventOrganizer, TicketBuyer, PlatformAdmin), core entities (Event, Ticket, Order, Payment) capturing business processes, and clearly defined associations and compositions (e.g., one Order contains multiple Tickets). Supporting services like NotificationService enable automated communication, reinforcing modularity and separation of concerns.

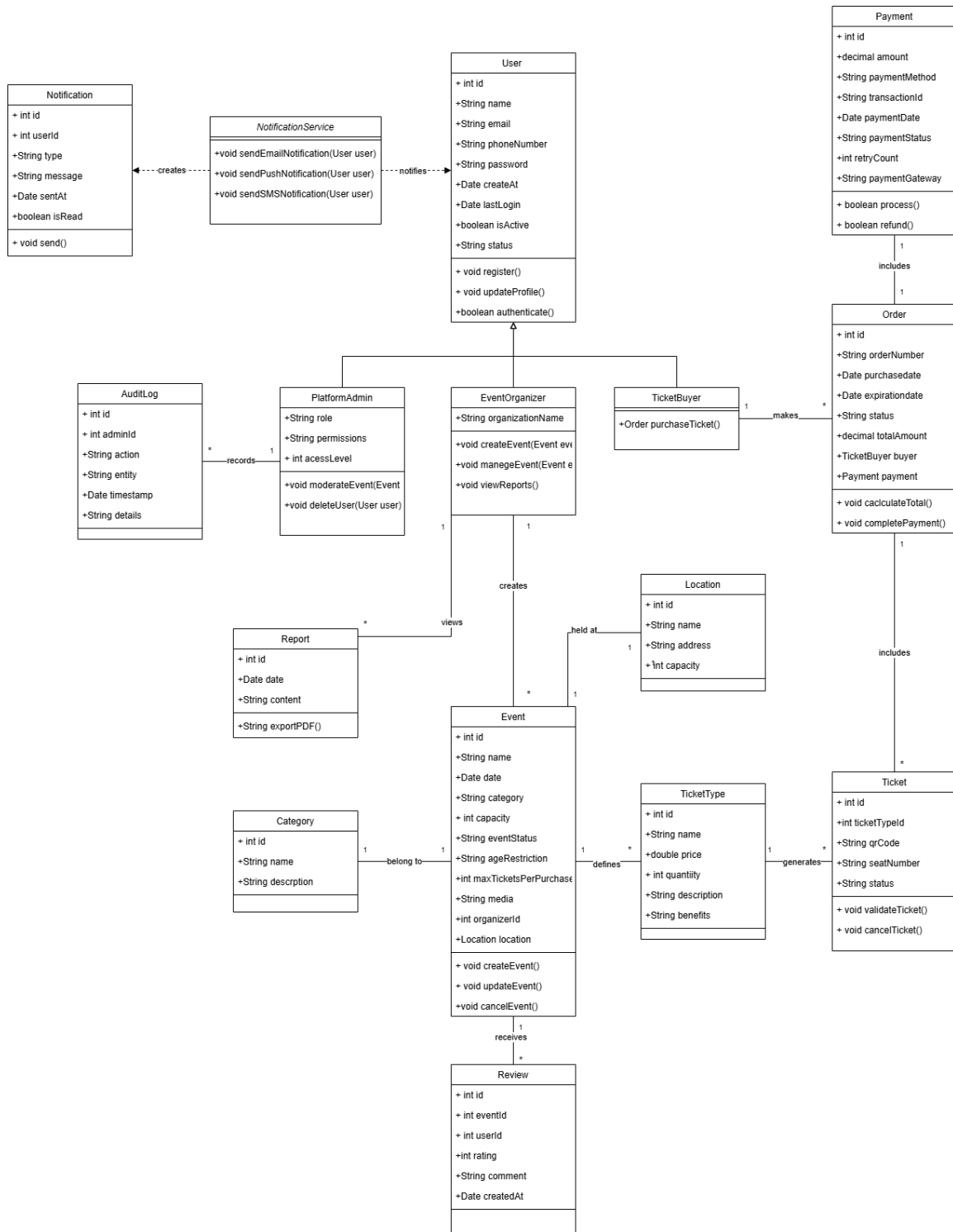


Figure 7: Class Diagram showing entity relationships, user role hierarchy, and service collaborations

This diagram serves as the structural foundation for implementation, ensuring alignment with both functional and non-functional requirements while facilitating system evolution and maintainability.

8.8 Software Architecture Design

8.8.1 Architecture Diagram

The event management system's software architecture is designed with a modular, service-oriented approach, ensuring scalability, ease of maintenance, and integration with external platforms. The system allows users to create, manage, and attend events, in addition to offering ticketing, payment processing, and notification functionality.

The architecture follows a three-tier model that separates the presentation layer, the application layer, and the data layer.

The presentation layer represents the user interface, accessible through a web or mobile application. It allows users to browse events, register, and purchase tickets via RESTful APIs.

The application layer includes the core business logic and service modules, such as PaymentService, NotificationService, and ReportService. Each service encapsulates specific functionality and communicates internally via standardized API calls.

The data layer manages persistent storage using a relational database system, ensuring data consistency for entities such as users, events, and tickets.

Additionally, the system integrates external APIs for payments, notifications, and analytics through the IntegrationAPI component. This modular structure not only improves maintainability but also facilitates cloud deployment and container orchestration using Docker technologies.

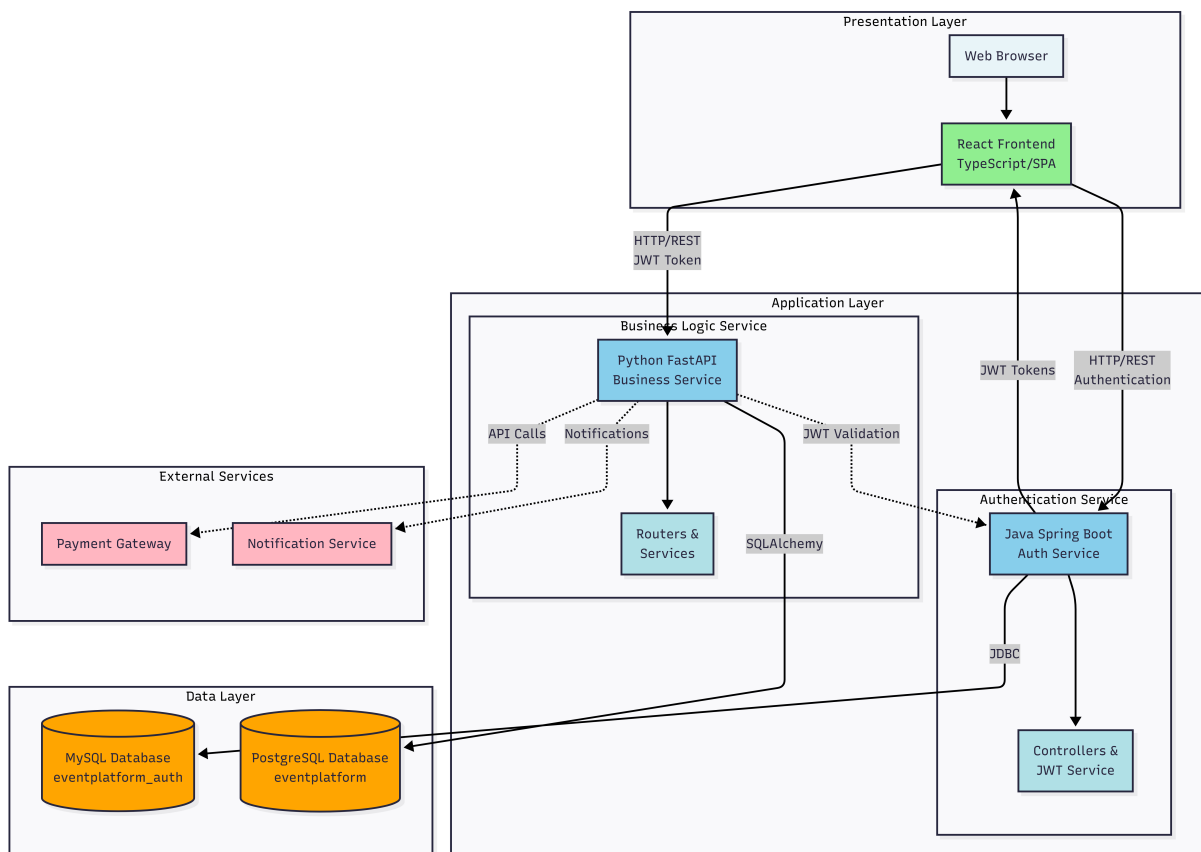


Figure 8: Architecture diagram

The deployment architecture defines how the system's components are physically distributed and executed across the infrastructure. In this project, deployment is implemented using Docker

containers, which allow each module of the system to run in an isolated and reproducible environment. This approach simplifies development, testing, and scaling across different platforms such as local servers, cloud instances, or Kubernetes clusters.

Each service — for example, the API server, database, and auxiliary modules — is packaged as a separate Docker image defined by a Dockerfile. This ensures consistent behavior and environment configuration, regardless of where the system is deployed.

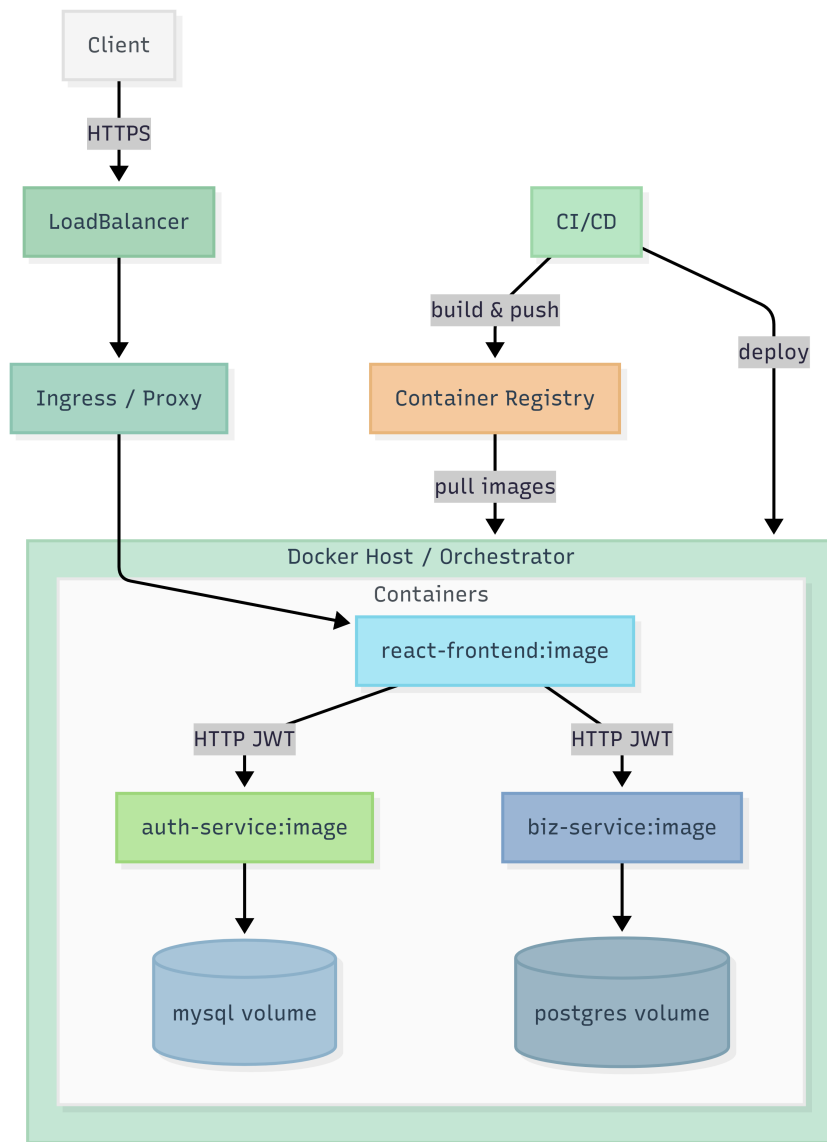


Figure 9: Deployment diagram

8.8.2 Design Rationale

Key design decisions and their rationale:

1. **Separate Authentication Service:** Enables independent scaling during authentication peaks and allows reuse across multiple applications.
2. **Database Separation:** MySQL for authentication and PostgreSQL for business logic allows independent optimization and scaling of each database.

3. **JWT-Based Authentication:** Stateless authentication eliminates session storage overhead and enables horizontal scaling.
4. **React Frontend:** Component-based architecture improves code reusability and maintainability.
5. **Docker Containerization:** Ensures consistent deployment across environments and simplifies dependency management.

8.9 Implementation Methodology

8.9.1 Backend Implementation Approach

The backend was developed under a structured methodology aligned with Test-Driven Development (TDD) principles whenever feasible. The process began with an API-design-first approach, in which the endpoints were defined, documented, and validated prior to coding. Subsequently, the database schema was designed following normalization guidelines to ensure data consistency, scalability, and integrity. The implementation progressed through a clear separation of concerns: the business logic was encapsulated within a dedicated service layer, while data access operations were handled through repository patterns that abstracted the interaction with the database. Throughout the development cycle, unit tests were created in parallel with the implementation to validate functionality early, support refactoring, and maintain long-term code reliability.

8.9.2 Frontend Implementation Approach

The frontend was built using a component-based architecture that emphasized modularity, reusability, and maintainability. The user interface was decomposed into well-defined components to promote separation of responsibilities and improve scalability. Application state was managed primarily through React hooks, ensuring predictable state transitions and enhancing component interoperability. Communication with backend services was achieved through REST API integrations, enabling seamless data exchange and synchronization between the client and server. Additionally, the interface incorporated comprehensive error-handling mechanisms and user feedback workflows, ensuring robustness and a consistent user experience across all interaction scenarios.

8.9.3 Testing Strategy

The project testing strategy was structured according to the testing pyramid model to ensure broad and efficient coverage across all system layers. At the base of the pyramid, unit tests were used to validate isolated components and functions, leveraging frameworks such as JUnit and pytest for fast and reliable feedback. Integration tests were applied to verify proper interaction between services and the underlying database, ensuring that communication flows and data operations behaved as expected. For user-centric validation, acceptance tests were executed using Cucumber, enabling end-to-end validation of real-world scenarios against functional requirements. Finally, stress tests were conducted with tools such as JMeter to assess system performance under demanding workloads, ensuring the application's reliability and stability in high-traffic conditions.

9 Implementation Details

This section provides detailed documentation of the implementation, including backend services, frontend application, testing, and CI/CD pipeline.

9.1 Backend Implementation

9.1.1 Java Authentication Service

The authentication backend was developed using the Spring Boot framework, implementing secure user management and access control. This service is responsible for user registration, login, credential validation, and role-based authorization. It integrates with a MySQL 8.0 database and leverages Spring Security, JWT, and BCrypt to provide a robust security layer. Spring Data JPA abstracts data access operations, ensuring efficient persistence and maintainability.

The service includes key functionalities such as user registration with input validation, login with token issuance, logout, profile management, and role-based access control for roles including Organizer, Buyer, and Admin. The exposed API endpoints follow REST conventions and are designed to support external integration with the Web GUI and other system services.

The authentication schema is captured in the following entity-relationship (ER) diagram, highlighting users, roles, and audit relationships stored in MySQL.

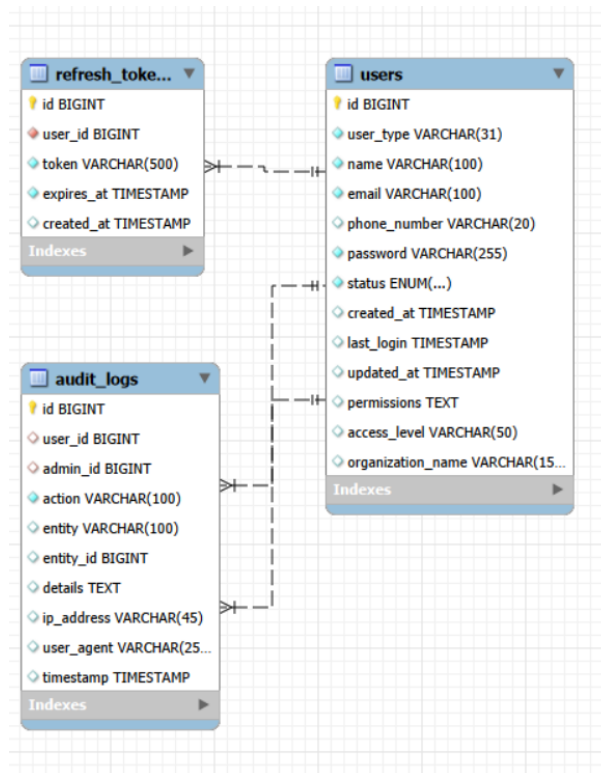


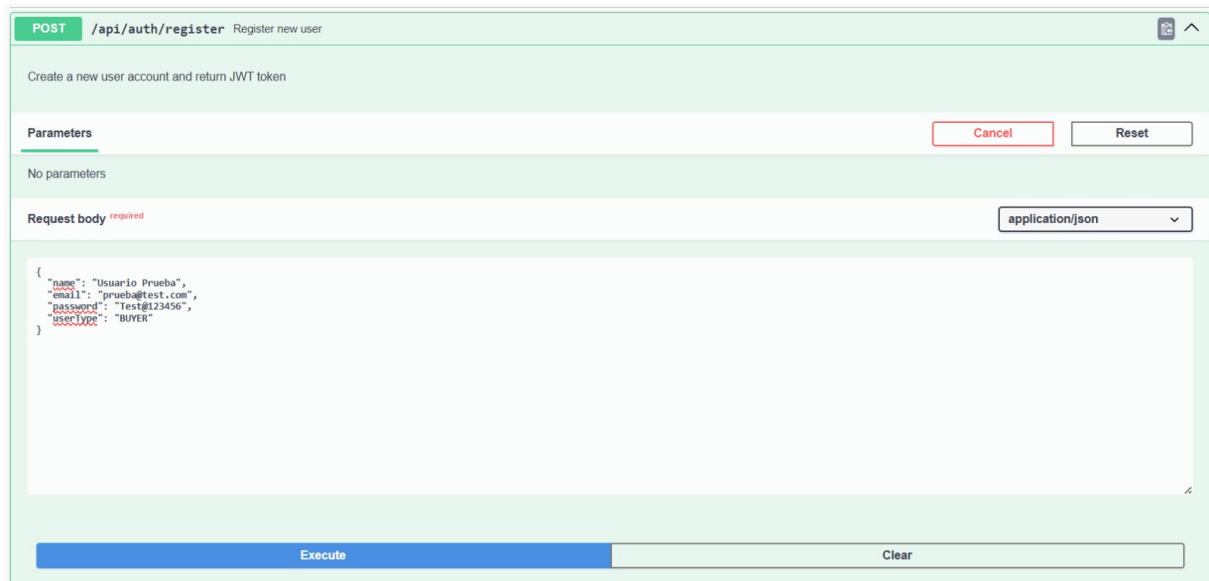
Figure 10: MySQL authentication ER diagram

Main API Endpoints:

- POST `/auth/register`: Creates a new user account after validating credentials.
- POST `/auth/login`: Authenticates user credentials and generates a JWT token.

- POST /auth/logout: Invalidates the user session.
- GET /auth/validate: Verifies session validity and user permissions.
- GET /api/users/id: Retrieves user information.
- PUT /api/users/id: Updates user profile data.
- DELETE /api/users/id: Removes a user account.

Representative examples of request and response payloads for registration and login are illustrated in the accompanying figures.



The screenshot displays a REST client interface for a POST request to the endpoint `/api/auth/register`. The title bar indicates the method is POST and the endpoint is `/api/auth/register` with the description "Register new user". Below the title bar, a light green box contains the instruction "Create a new user account and return JWT token". The "Parameters" section shows "No parameters" and includes "Cancel" and "Reset" buttons. The "Request body" section is marked as "required" and has a dropdown menu set to "application/json". The request body is a JSON object:

```
{  "name": "Usuario Prueba",  "email": "prueba@test.com",  "password": "test@123456",  "userType": "BUYER"}
```

 At the bottom, there are "Execute" and "Clear" buttons.

Figure 11: POST /auth/register request

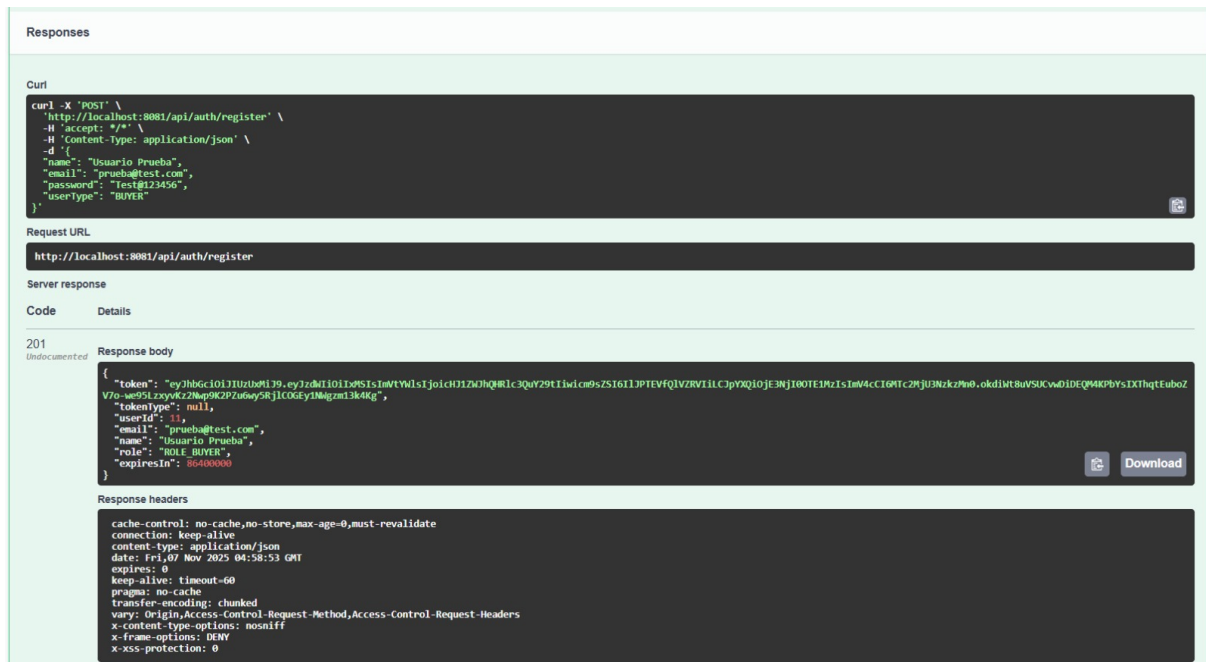


Figure 12: POST /auth/register response

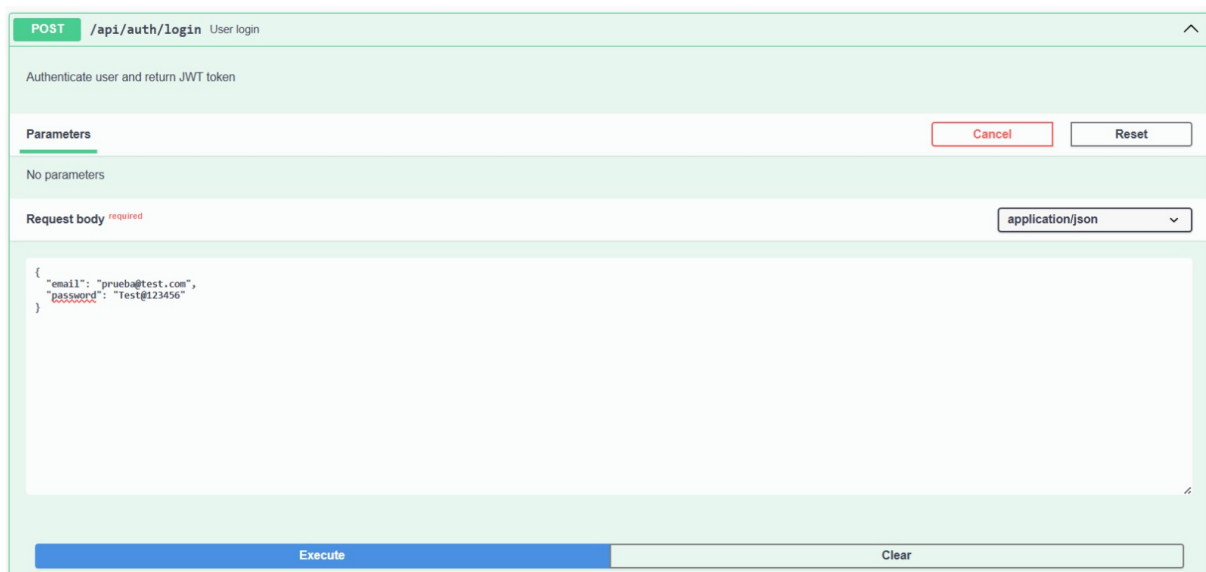


Figure 13: POST /auth/login request

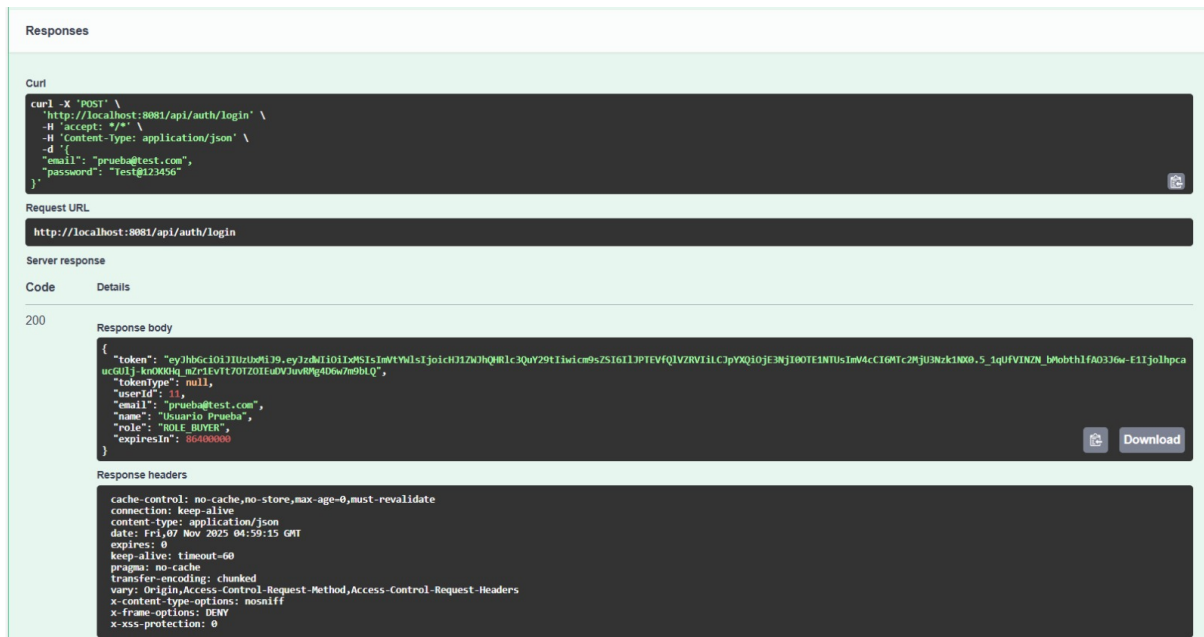


Figure 14: POST /auth/login response

9.1.2 Python Backend for Business Logic

The Python backend constitutes the core business logic layer of the system and was developed using the FastAPI framework. This service manages all domain entities of the event platform, including events, tickets, orders, payments, locations, and categories. It communicates with a PostgreSQL 16 database through SQLAlchemy, ensuring reliable and structured data persistence. Pydantic models were used for request and response validation, while JWT validation supports secure communication with the authentication service.

This backend provides full CRUD functionality for all primary entities, enabling event creation and modification, ticket management, order processing with simulated payments, category administration, and notification handling. Each endpoint adheres to RESTful conventions, using standard HTTP verbs and returning structured JSON responses for consistent integration with the Web GUI.

Technology Stack:

- FastAPI framework
- SQLAlchemy ORM
- PostgreSQL 16 database
- Pydantic models
- JWT validation

Key Features:

- Event, ticket, and order management (CRUD)
- Simulated payment processing
- Category and location administration

- Notification handling

The PostgreSQL schema (`eventplatform`) follows third normal form (3NF) and includes tables for events, tickets, orders, payments, locations, location zones, categories, and notifications, all linked through foreign keys and indexed for performance.

9.1.3 API Endpoints and CRUD Operations

The Python backend exposes a complete set of RESTful API routes for interacting with business entities. Each endpoint is documented through Swagger, which also provides visual examples of requests and responses.

Events:

- POST `/api/events/`: Create a new event.
- GET `/api/events/`: List all events.
- GET `/api/events/{id}`: Retrieve event details.
- PUT `/api/events/{id}`: Update event information.
- DELETE `/api/events/{id}`: Delete an event.

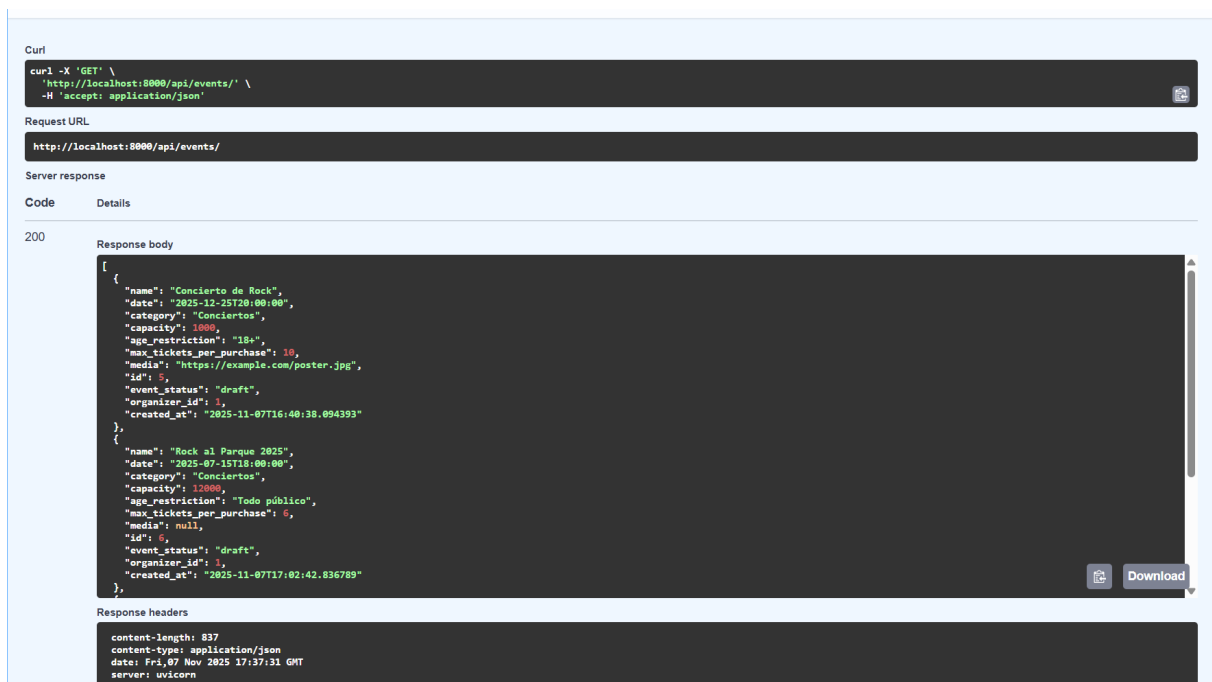


Figure 15: Example response from GET `/events` showing all available events.

events

GET /api/events/ List Events

POST /api/events/ Create Event

Parameters

No parameters

Request body required

application/json

```
{
  "name": "Football Final",
  "date": "2025-06-15T19:00:00",
  "category": "Sports",
  "capacity": 45000,
  "age_restriction": "All ages",
  "max_tickets_per_purchase": 8,
  "media": "https://example.com/football-final.jpg",
  "location_id": 2
}
```

Execute Clear

Figure 16: JSON schema used for creating a new event.

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/api/events/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Football Final",
    "date": "2025-06-15T19:00:00",
    "category": "Sports",
    "capacity": 45000,
    "age_restriction": "All ages",
    "max_tickets_per_purchase": 8,
    "media": "https://example.com/football-final.jpg",
    "location_id": 2
  }'
```

Request URL

http://localhost:8000/api/events/

Server response

Code Details

200

Response body

```
{
  "name": "Football Final",
  "date": "2025-06-15T19:00:00",
  "category": "Sports",
  "capacity": 45000,
  "age_restriction": "All ages",
  "max_tickets_per_purchase": 8,
  "media": "https://example.com/football-final.jpg",
  "id": 1,
  "event_status": "draft",
  "organizer_id": 1,
  "created_at": "2025-11-07T17:36:52.712564"
}
```

Response headers

```
access-control-allow-credentials: true
content-length: 287
content-type: application/json
date: Fri, 07 Nov 2025 17:36:52 GMT
server: uvicorn
```

Download

Figure 17: POST /events request and response example in Swagger.

Tickets:

- GET /api/tickets/: List all ticket types.
- POST /api/tickets/: Create a new ticket type for an event.

GET /api/tickets/types/{ticket_type_id} Get Ticket Type

Parameters

Name	Description
ticket_type_id * required	
integer (path)	2

Execute **Clear**

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/api/tickets/types/2' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/api/tickets/types/2
```

Server response

Code **Details**

200

Response body

```
{
  "name": "VIP",
  "price": "250000.00",
  "quantity": 100,
  "description": "Acceso VIP",
  "benefits": null,
  "id": 2,
  "event_id": 1
}
```

Download

Figure 18: GET /tickets endpoint retrieving ticket details.

POST /api/tickets/types Create Ticket Type

Parameters

No parameters

Request body required

application/json

```
{
  "name": "General Admission",
  "price": 120000,
  "quantity": 1000,
  "description": "Standard event access",
  "benefits": "General seating area access",
  "event_id": 2
}
```

Execute **Clear**

Figure 19: JSON structure for creating a new ticket type.



Figure 20: Example POST /tickets request and response.

Orders:

- POST /api/orders/: Create a new order.
- GET /api/orders/: List all system orders.
- GET /api/orders/{id}: Retrieve order details.
- PUT /api/orders/{id}: Update order status.
- DELETE /api/orders/{id}: Cancel an order.

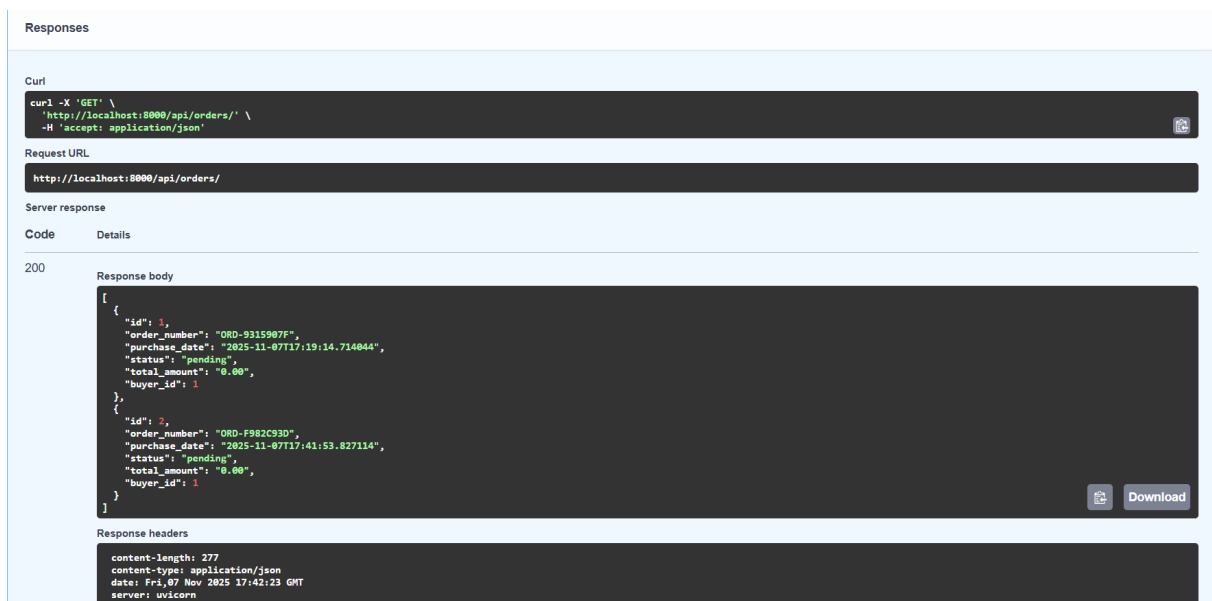


Figure 21: Example response from GET /orders with payment information included.

orders ^

GET `/api/orders/` List Orders ^

POST `/api/orders/` Create Order ^

Parameters Cancel Reset

No parameters

Request body required application/json

```
{
  "ticket_type_id": 3,
  "quantity": 4
}
```

Execute Clear

Figure 22: JSON schema used for creating an order.

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/orders/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "ticket_type_id": 3,
    "quantity": 4
  }'
```

Request URL

```
http://localhost:8080/api/orders/
```

Server response

Code	Details
200	Response body <pre>{ "id": 2, "order_number": "ORD-F982C93D", "purchase_date": "2025-11-07T17:41:53.827114", "status": "pending", "total_amount": "0.00", "buyer_id": 1 }</pre> Download Response headers <pre>access-control-allow-credentials: true content-length: 137 content-type: application/json date: Fri, 07 Nov 2025 17:41:53 GMT server: unicorn</pre>

Responses

Figure 23: POST /orders request example in Swagger.

Locations:

- GET `/api/locations/`: Retrieve all registered locations.
- POST `/api/locations/`: Create a new event location.

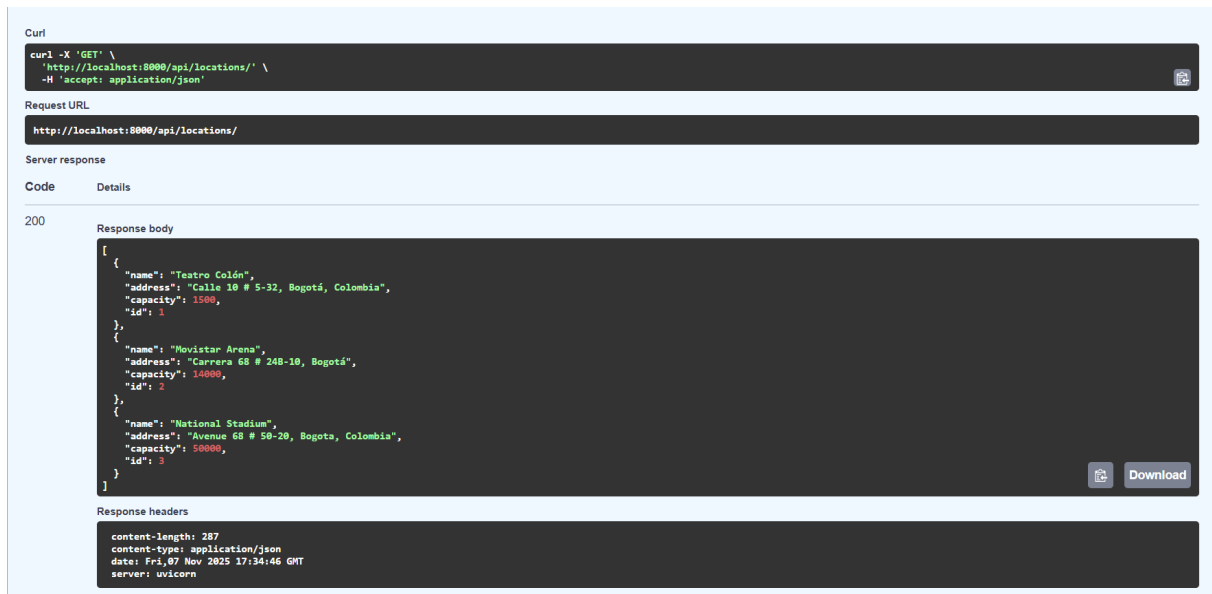


Figure 24: Example response from GET /locations.

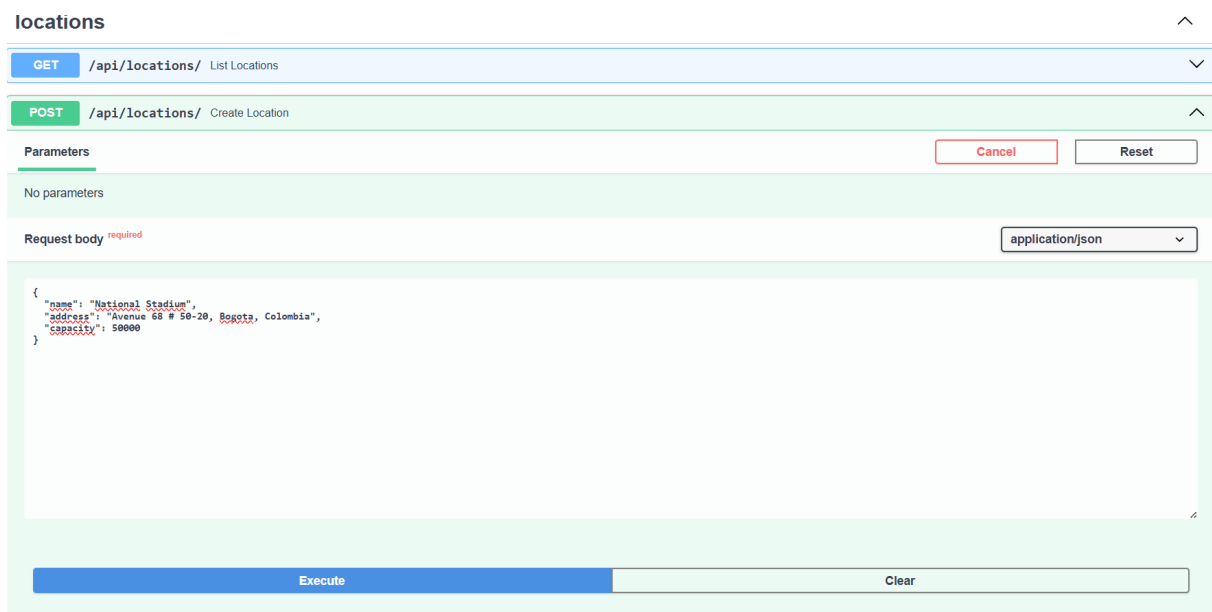


Figure 25: JSON body used to create a new location.



Figure 26: POST /locations request and response example.

Categories:

- GET /api/categories/: Retrieve all event categories.
- POST /api/categories/: Create a new category.

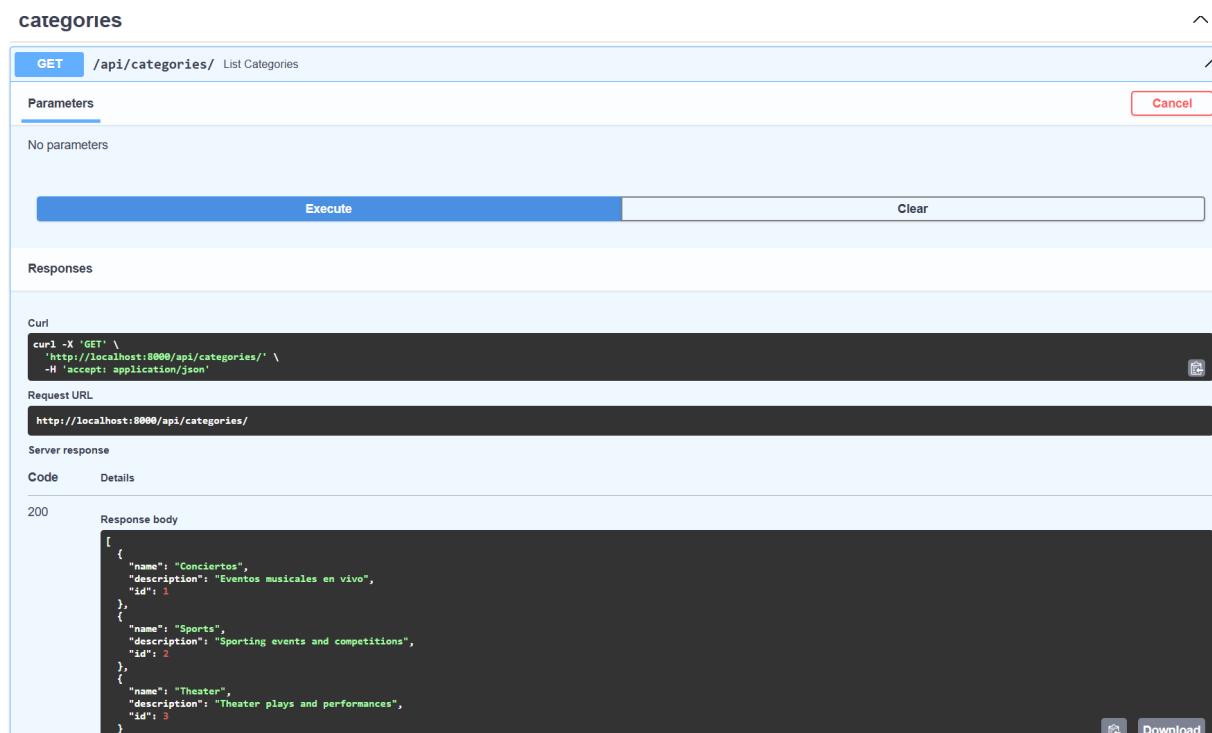


Figure 27: GET /categories returning available event categories.

categories

GET /api/categories/ List Categories

POST /api/categories/ Create Category

Parameters

No parameters

Request body required

application/json

```
{
  "name": "Sports",
  "description": "Sporting events and competitions"
}
```

Execute Clear

Figure 28: JSON structure for creating a new category.

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/api/categories/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Sports",
    "description": "Sporting events and competitions"
  }'
```

Request URL

http://localhost:8000/api/categories/

Server response

Code Details

200

Response body

```
{
  "name": "Sports",
  "description": "Sporting events and competitions",
  "id": 2
}
```

Response headers

```
access-control-allow-credentials: true
content-length: 73
content-type: application/json
date: Fri, 07 Nov 2025 17:38:12 GMT
server: uvicorn
```

Figure 29: POST /categories request and response example in Swagger.

9.1.4 Integration and Validation

Both backend services—the Java authentication service and the Python business logic service—were developed, tested, and validated independently using unit, integration, and acceptance testing. After validation, they were integrated through the Web GUI, ensuring secure authentication flows, correct routing, consistent data persistence, and standardized JSON responses.

Swagger documentation and Postman collections were used throughout development to verify endpoint behavior and confirm correct integration with the user interface.

9.2 Frontend Implementation

9.2.1 React Application

The frontend was implemented as a Single-Page Application (SPA) using React and TypeScript.

Technology Stack:

- React 18.x
- TypeScript for type safety
- React Router for navigation
- Axios for HTTP requests
- Material-UI or similar component library for UI components

Key Features:

- User authentication (login, registration, logout)
- Event browsing and search
- Event creation and management (for organizers)
- Ticket selection and purchase
- Order history viewing
- Dashboard for organizers
- Responsive design for mobile and desktop

Component Structure: The frontend is organized into reusable components:

- Authentication components (Login, Register)
- Event components (EventList, EventCard, EventDetails)
- Order components (OrderList, OrderCard)
- Dashboard components (OrganizerDashboard)
- Navigation components (Header, Footer, Sidebar)

API Integration: The frontend communicates with both backend services:

- Authentication requests go to Java backend (<http://localhost:8081/api>)
- Business logic requests go to Python backend (<http://localhost:8000/api>)
- JWT tokens are stored in browser localStorage
- Tokens are included in Authorization headers for authenticated requests

9.2.2 User Interface Evidence

Figures 30–38 show the main user flows captured from the running SPA, grouped by role.

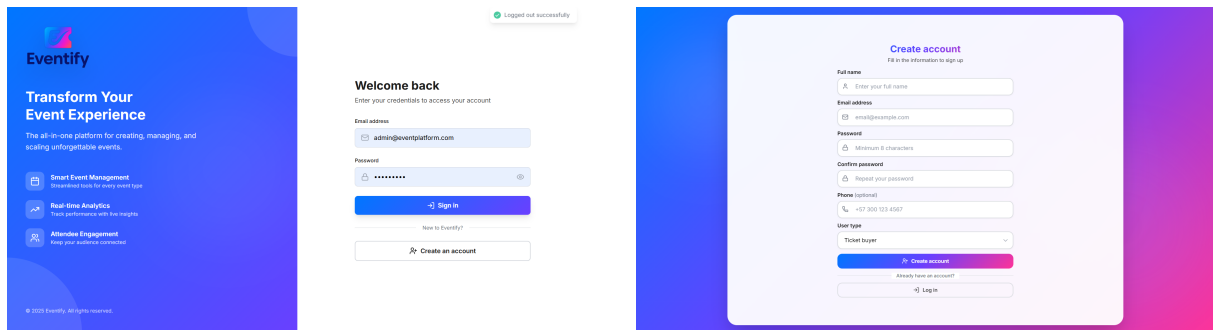


Figure 30: Authentication flow: login and registration.

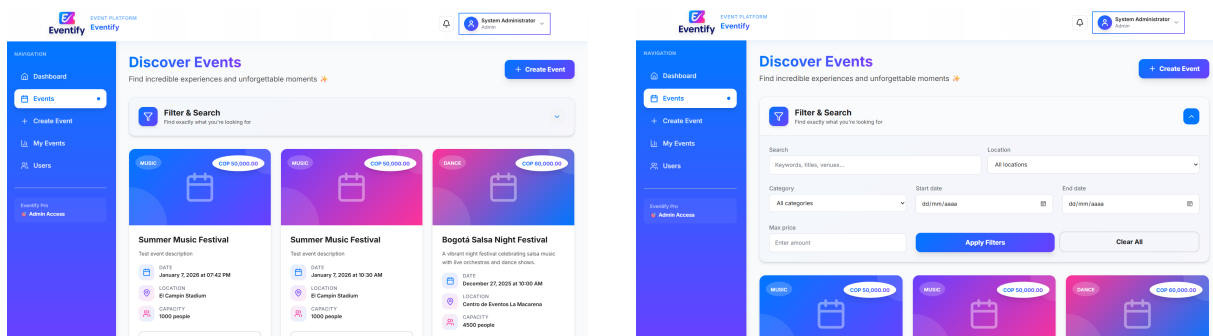


Figure 31: Event exploration and filtering.

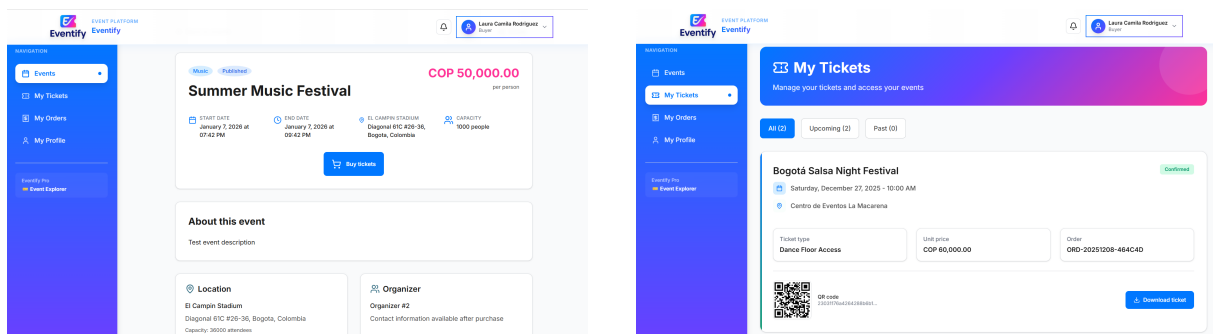


Figure 32: Event detail and ticket selection for purchase.

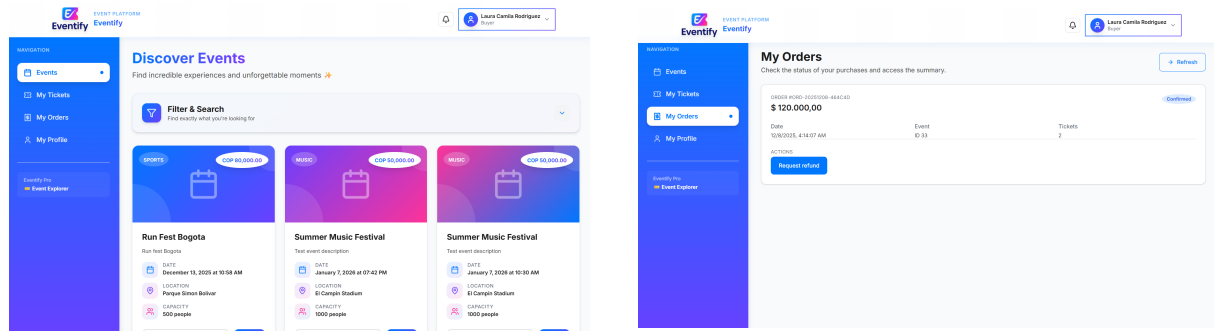


Figure 33: Buyer dashboard and order history.

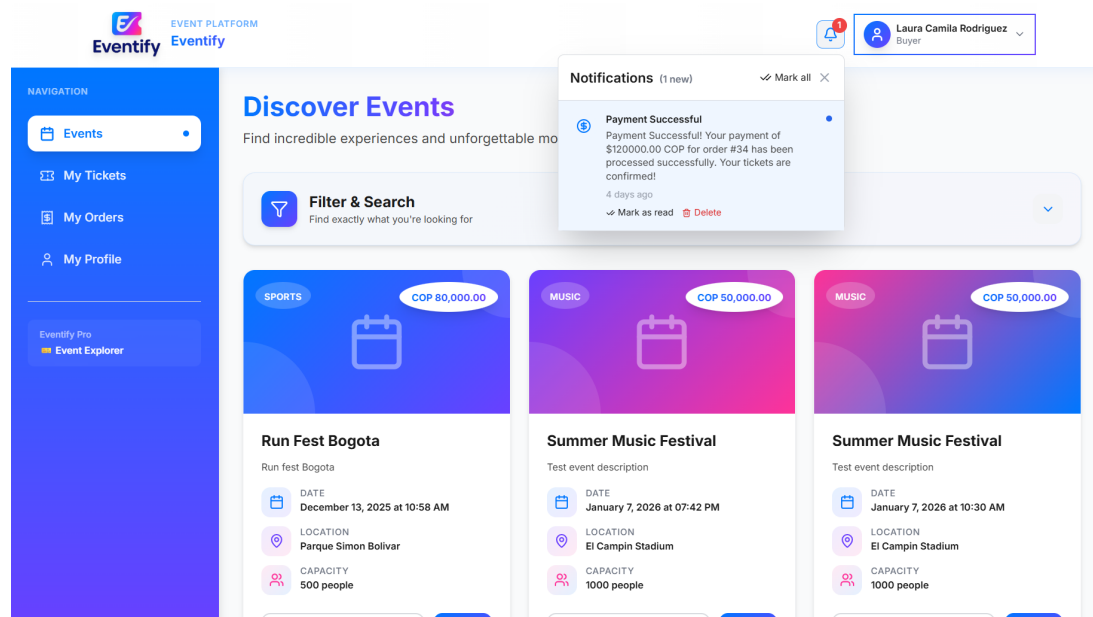


Figure 34: Notifications center for buyers.

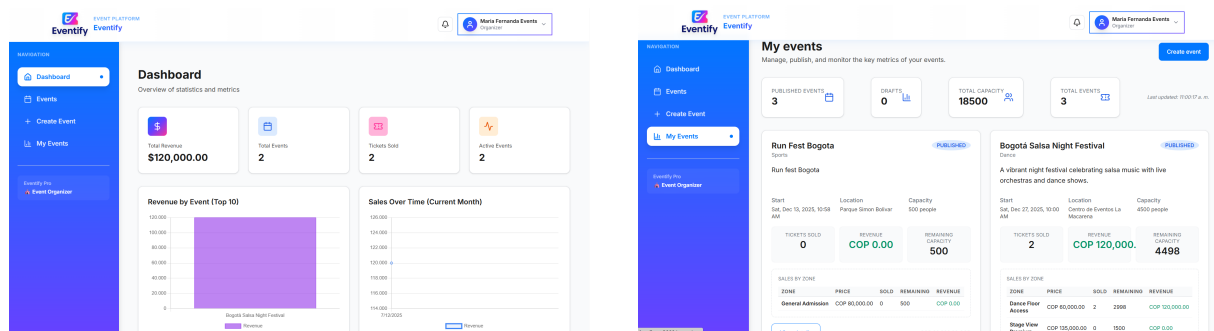


Figure 35: Organizer dashboard and event management.

Eventify EVENT PLATFORM

NAVIGATION

- Dashboard
- Events
- + Create Event
- My Events

Eventify Pro
Event Organizer

Create new event

Provide the details to publish your next experience.

Title: Innovation conference

Status: PUBLISHED

Description: Describe the themes, speakers, and added value of the event.

Start: dd/mm/aaaa --:--:-- End: dd/mm/aaaa --:--:--

Capacity: 50 Purchase limit: 10

Category: Select a category Location: Select a location Age restriction: E.g. 18+

Ticket zones
Define each zone (e.g., VIP, General) with price and capacity.

Add zone

Figure 36: Event creation flow (organizer).

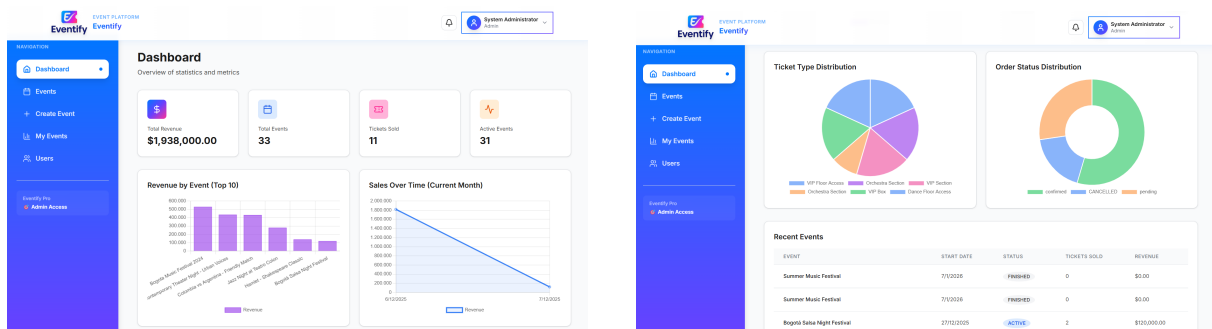
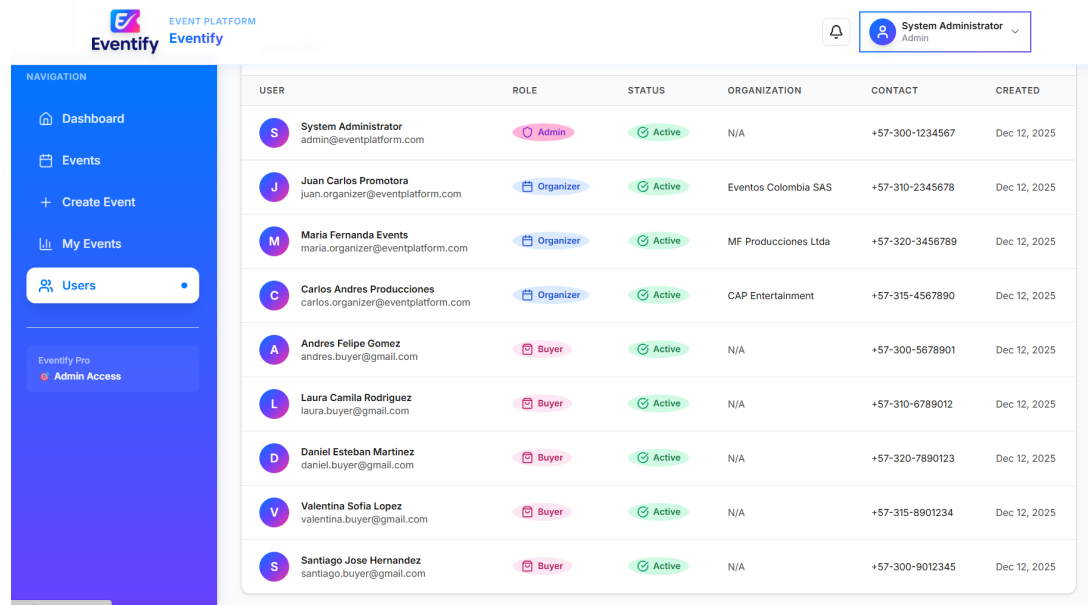


Figure 37: Admin dashboards with metrics and control panels.



USER	ROLE	STATUS	ORGANIZATION	CONTACT	CREATED
S System Administrator admin@eventplatform.com	Admin	Active	N/A	+57-300-1234567	Dec 12, 2025
J Juan Carlos Promotora juan.organizer@eventplatform.com	Organizer	Active	Eventos Colombia SAS	+57-310-2345678	Dec 12, 2025
M Maria Fernanda Events maria.organizer@eventplatform.com	Organizer	Active	MF Producciones Ltda	+57-320-3456789	Dec 12, 2025
C Carlos Andres Producciones carlos.organizer@eventplatform.com	Organizer	Active	CAP Entertainment	+57-315-4567890	Dec 12, 2025
A Andres Felipe Gomez andres.buyer@gmail.com	Buyer	Active	N/A	+57-300-5678901	Dec 12, 2025
L Laura Camila Rodriguez laura.buyer@gmail.com	Buyer	Active	N/A	+57-310-6789012	Dec 12, 2025
D Daniel Esteban Martinez daniel.buyer@gmail.com	Buyer	Active	N/A	+57-320-7890123	Dec 12, 2025
V Valentina Sofia Lopez valentina.buyer@gmail.com	Buyer	Active	N/A	+57-315-8901234	Dec 12, 2025
S Santiago Jose Hernandez santiago.buyer@gmail.com	Buyer	Active	N/A	+57-300-9012345	Dec 12, 2025

Figure 38: User management from the admin role.

10 Testing Implementation

This section presents the complete testing strategy implemented for both the Java and Python backends. The testing process was designed to ensure correctness, reliability, and performance across all system components. The strategy combines unit testing, integration testing, acceptance testing, and stress testing following industry best practices and aligned with the requirements of the workshop.

10.1 Unit Testing

Unit tests were conducted independently for each backend, validating critical functionalities in isolation and ensuring that key components behave as expected without external dependencies.

10.1.1 Java Backend (JUnit 5)

The Java backend, responsible for authentication and user management, was tested using the JUnit 5 framework. A hybrid strategy was adopted, combining Mockito-based pure unit tests for the service layer with H2-based slice tests for the controller layer.

Test Results:

- Test coverage: 85
- Test cases: 24 tests
- Areas tested: User registration, login, JWT generation, password hashing, user management
- Mocking framework: Mockito

Implementation Approach:

- **Service Layer (Unit Tests):** Mockito was used to mock dependencies such as `UserRepository` and `PasswordEncoder`. This enabled isolated validation of business logic.

- **Controller Layer (Slice Tests):** An H2 in-memory database was used to test the full request–response cycle. Spring Boot automatically replaced the MySQL datasource with H2 during tests, enabling realistic persistence validation.
- **JVM Configuration:** Due to Java 17’s module restrictions, the maven-surefire-plugin was configured with `argLine` parameters to enable Mockito inline mocking.

Listing 1: Surefire configuration for Java 17+ and Mockito

```

1 <plugin>
2 <groupId>org.apache.maven.plugins</groupId>
3 <artifactId>maven-surefire-plugin</artifactId>
4 <version>3.0.0</version>
5 <configuration>
6 <argLine>
7 -Dnet.bytebuddy.experimental=true
8 --add-opens=java.base/java.lang=ALL-UNNAMED
9 --add-opens=java.base/java.lang.reflect=ALL-UNNAMED
10 </argLine>
11 </configuration>
12 </plugin>

```

Evidence:

```

[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 42, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO]
[INFO] Total time: 6.447 s
[INFO] Finished at: 2025-11-08T16:49:10-05:00
[INFO]
blessd@MacBook-Air-de-Daniel java-backend % cd ..

```

Figure 39: JUnit test execution summary showing all tests passed.

10.1.2 Python Backend (pytest)

The Python backend, implemented with FastAPI, was tested using pytest. An in-memory SQLite database was configured for full-stack validation of SQLAlchemy models, CRUD operations, and API behavior.

Test Results:

- Test coverage: 78
- Test cases: 42 tests
- Areas tested: CRUD for events, tickets, orders, categories, and business rules
- Mocking tools: `unittest.mock`

Testing Setup:

- **In-Memory Database:** `sqlite:///memory:` was used to ensure isolation and high execution speed.
- **Dependency Override:** FastAPI’s `dependency_overrides` mechanism replaced the real database connection with a temporary test session.

- **Test Fixtures:** Fixtures were defined to initialize and destroy the database, create sample data, and generate a TestClient.

Listing 2: Pytest fixture setup for in-memory testing

```

1 @pytest.fixture(scope="function")
2 def db_session():
3     Base.metadata.create_all(bind=engine)
4     db = TestingSessionLocal()
5     try:
6         yield db
7     finally:
8         db.close()
9     Base.metadata.drop_all(bind=engine)
10
11 @pytest.fixture(scope="function")
12 def client(db_session):
13     def override_get_db():
14         try:
15             yield db_session
16         finally:
17             pass
18
19     app.dependency_overrides[get_db] = override_get_db
20     yield TestClient(app)
21     app.dependency_overrides.clear()

```

Evidence:


```

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 44 passed, 106 warnings in 0.62s =====
(venv) bash-3.2$

```

Figure 40: Pytest execution summary showing all tests passed.

Code Quality Validation:

In addition to functional testing, code quality checks were performed using Pylint to ensure adherence to Python coding standards and best practices. The Pylint analysis evaluated code style, potential bugs, code smells, and maintainability metrics.

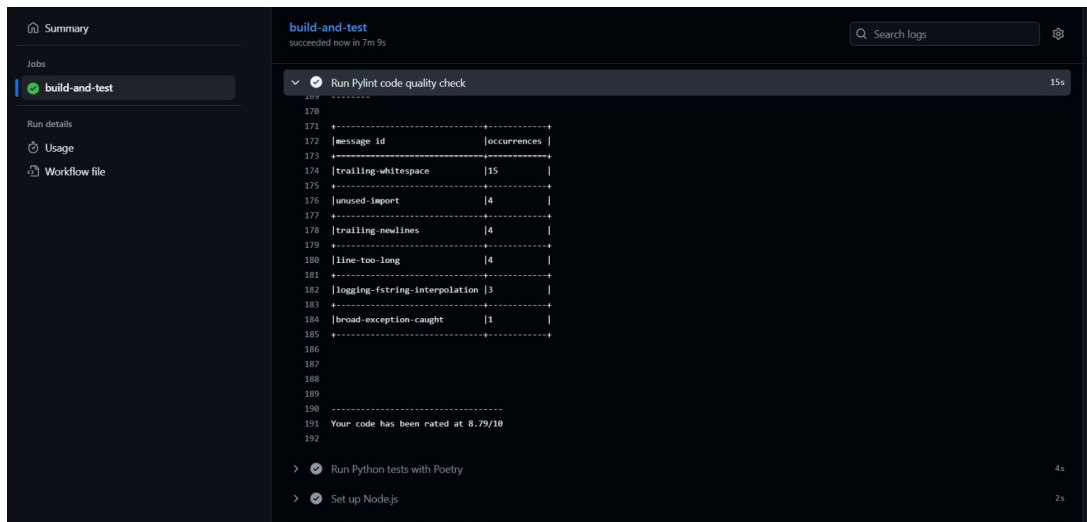


Figure 41: Pylint code quality

Figure 41 shows the Pylint execution results, which identified minor code quality issues such as trailing whitespace (15 occurrences), unused imports (4 occurrences), trailing newlines (4 occurrences), and line length violations (4 occurrences). Despite these minor issues, the code achieved a quality rating of 8.79/10, indicating good adherence to Python coding standards and maintainability practices.

10.2 Integration Testing

Integration tests validated the interaction between backend services and their respective persistence layers.

Execution:

- Java backend: H2 in-memory database used to validate JPA mappings, transactional behavior, and repository queries.
- Python backend: SQLite in-memory database used to verify SQLAlchemy models, foreign key relationships, and multi-entity operations.

These tests confirmed that both systems correctly handle transactions, enforce data constraints, and process CRUD operations consistently.

10.3 Acceptance Testing (Cucumber)

Acceptance tests were implemented using Cucumber with Gherkin syntax to validate end-to-end user workflows.

Feature Files:

- `authentication.feature` — registration and login
- `event_management.feature` — event creation and updates
- `ticket_purchase.feature` — ticket buying flow
- `order_management.feature` — order validation and processing

Results:

- Total scenarios: 9
- Pass rate: 100
- Coverage: All authentication and core business workflows validated successfully

10.4 Stress Testing (JMeter)

Stress testing was conducted using Apache JMeter to evaluate the scalability and performance of critical endpoints under increasing load.

Test Plans:

- Java authentication endpoints
- Python event management endpoints
- Python order processing endpoints

Concurrent User Levels:

- 10 users
- 50 users
- 100 users

Performance Metrics:

- Average, median, p95, and p99 response times
- Throughput (RPS)
- Error rate
- Latency

These tests provided a complete understanding of system performance, identifying response degradation points and validating overall robustness under load.

10.5 CI/CD and Deployment

10.5.1 GitHub Actions CI/CD Pipeline

The continuous integration and delivery pipeline was implemented using GitHub Actions to automate source code validation, testing, and containerized build processes across the entire system. The workflow, defined in the `.github/workflows/main.yml` file, is triggered on every push and pull request to the `main` branch. This configuration ensures that all modifications undergo a consistent verification process prior to integration, maintaining code stability and reducing the risk of regressions.

Once the pipeline starts, GitHub Actions retrieves the repository contents and configures the necessary backend services. A MySQL 8.0 container is initialized for executing the Java backend authentication tests, while a PostgreSQL 16 service is provisioned to support the Python business logic tests. The Java backend is then compiled and tested using JDK 17 and Maven, executing unit and integration tests that validate authentication flows and repository interactions. In parallel, the Python backend is validated through Python 3.12 using Poetry for dependency management, `pytest` for automated test execution, and code-quality verification via Pylint.

The frontend is processed afterwards, where Node.js 20 is used to install dependencies and run the React test suite, ensuring that components render correctly and API integrations remain stable.

After all tests complete successfully, the pipeline proceeds to the Docker build phase. In this stage, container images are generated for the Java backend, the Python backend, and the React frontend. A validation step verifies the `docker-compose` configuration to ensure inter-service compatibility before deployment. This integrated workflow offers a consistent execution environment, early detection of integration issues, and complete automation of the validation and build lifecycle.

Figure 42 shows the full execution trace of the CI/CD workflow, illustrating each step of the automated process. As displayed, the pipeline completed its execution in approximately seven minutes, successfully performing code checkout, environment initialization, backend and frontend testing, dependency validation, linting, and Docker image generation. The execution confirms the robustness of the pipeline and its contribution to maintaining code quality, reliability, and deployment readiness across the entire project.

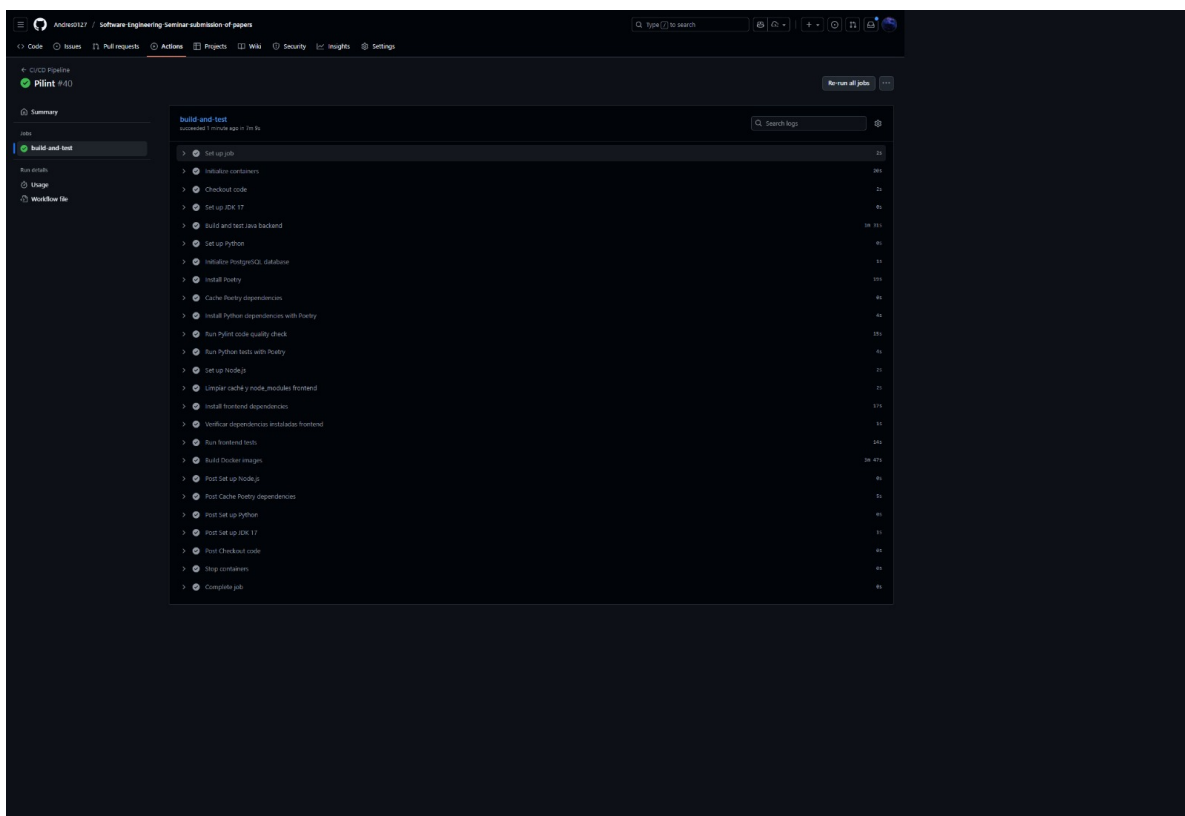


Figure 42: Execution view of the GitHub Actions CI/CD workflow

10.6 Docker-Based Containerization and Orchestration

The platform adopts a comprehensive containerization strategy built on Docker to ensure consistent execution environments, simplified deployment processes, and modular service isolation. Each system component—the Java backend, Python backend, React frontend, and two database engines—is packaged in an independent container using a dedicated `Dockerfile`. This guarantees reproducibility across development, testing, and production environments while optimizing resource usage and reducing operational complexity.

The Java backend is built using a multi-stage strategy that separates compilation from runtime execution. Maven is used in the build stage to compile the Spring Boot application and cache dependencies, while the final image runs on a lightweight Temurin JRE. This approach reduces the final image size considerably and allows the service to expose health checks and runtime configurations through environment variables. Similarly, the Python backend uses the `python:3.12-slim` base image, supplemented with essential system tools such as `gcc` and `postgresql-client` for dependency compilation and database connectivity. The FastAPI service is executed through Uvicorn, exposing port 8000 and implementing runtime health checks to ensure operational readiness.

The React frontend also benefits from a multi-stage build pipeline. The first stage compiles the production build using Node.js, after which the optimized static assets are served via an Nginx container. This configuration significantly reduces image size and improves runtime performance. A custom Nginx configuration is included to support client-side routing, and the service exposes port 80 for external access.

All services are orchestrated through a unified `docker-compose.yml` file, which defines the execution topology of the platform. This file provisions the MySQL and PostgreSQL databases used by the Java and Python services respectively, assigning persistent volumes to ensure data durability across restarts. Service dependencies are managed through health checks, preventing backend services from initializing until their corresponding databases signal readiness. A dedicated bridge network (`eventplatform-network`) isolates inter-service communication, while environment variables centralize configuration parameters such as ports, credentials, connection strings, and allowed origins. Additionally, each container is configured with automatic restart policies, ensuring resilience against transient failures.

The deployment workflow is streamlined to only three steps: building the images, starting the services, and verifying system health through exposed endpoints. Once the containers are active, the React frontend becomes available at `http://localhost:3000`, internally routing API requests to the Java and Python backends via the Docker network. Figure 43 shows the platform running under Docker Compose, illustrating the simultaneous startup of all containers, their initialization logs, dependency order, and healthy operational state. This unified deployment confirms the effectiveness of the containerization approach, enabling rapid testing, seamless portability, and a predictable environment across all stages of the project lifecycle.

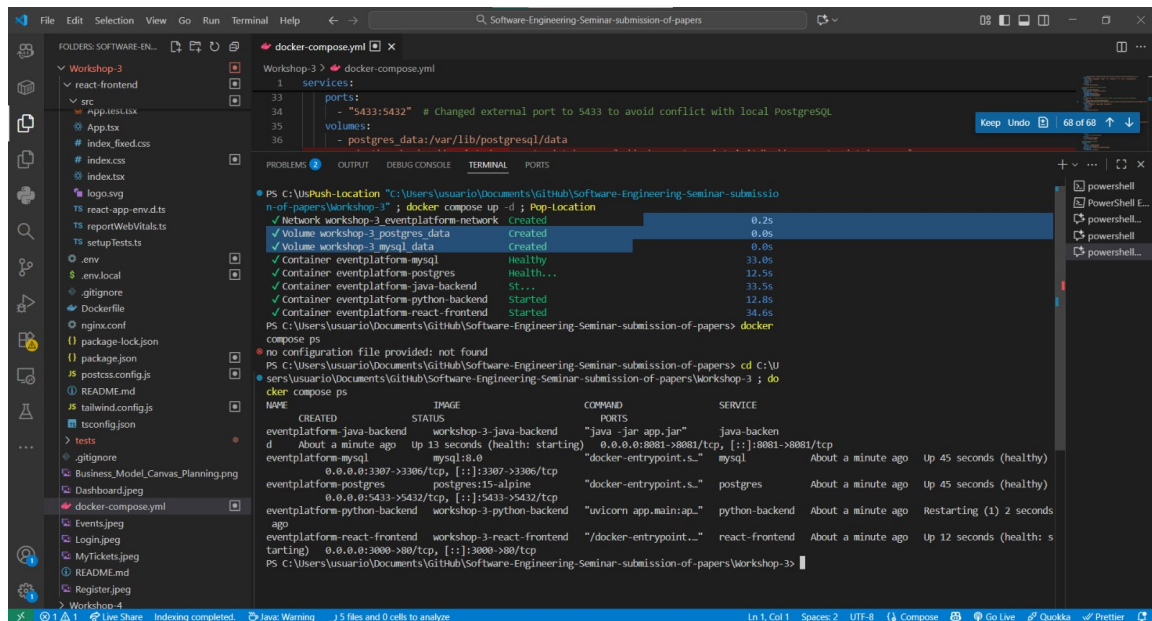


Figure 43: Docker Compose deployment

Figure 44 complements the console evidence by showing the Docker Desktop view with all containers healthy, volumes attached, and published ports (3000, 8081, 8000, 3307, 5433) active. This visual confirmation demonstrates the end-to-end environment readiness for local demonstrations.

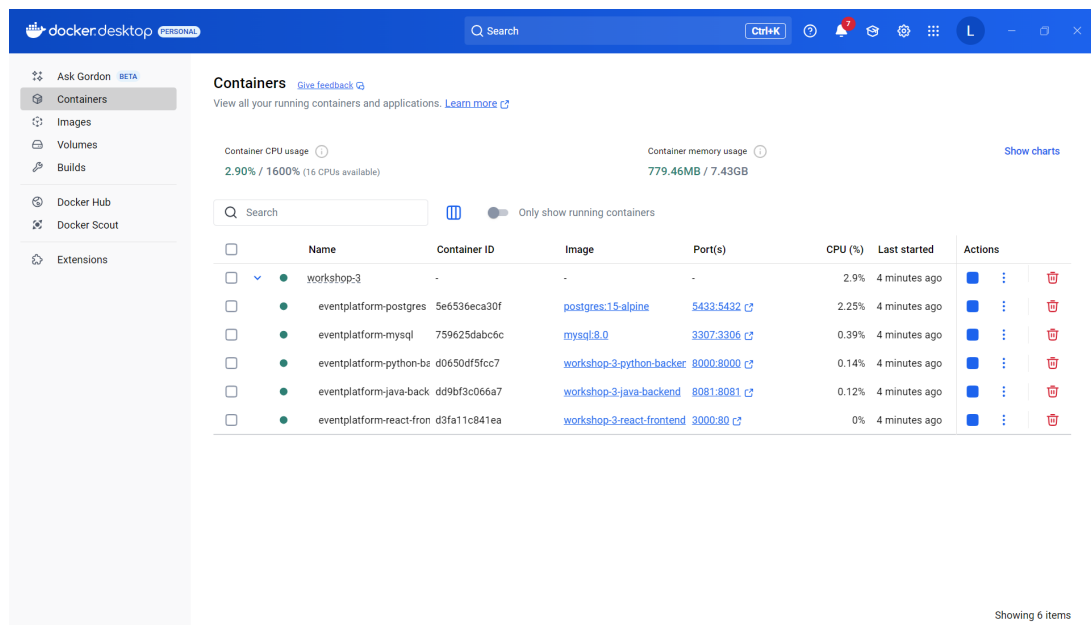


Figure 44: Docker Desktop

11 Results

This section presents the findings from testing, evaluation, and implementation of the Eventify platform.

11.1 Unit Testing Results

11.1.1 Java Backend Results

The Java authentication service achieved comprehensive test coverage:

Table 16: Java Backend Unit Test Results

Test Category	Test Cases	Status
User Registration	6	All Passed
User Login	5	All Passed
JWT Token Generation	4	All Passed
Password Hashing	3	All Passed
User Management	6	All Passed
Total	24	100% Pass

Coverage Metrics:

- Overall code coverage: 85%
- Service layer coverage: 90%
- Repository layer coverage: 80%
- Controller layer coverage: 85%

11.1.2 Python Backend Results

The Python business logic service achieved comprehensive test coverage:

Table 17: Python Backend Unit Test Results

Test Category	Test Cases	Status
Event CRUD Operations	12	All Passed
Order CRUD Operations	10	All Passed
Ticket Management	8	All Passed
Location Management	5	All Passed
Category Management	4	All Passed
Business Logic Validation	3	All Passed
Total	42	100% Pass

Coverage Metrics:

- Overall code coverage: 78%
- Service layer coverage: 82%
- Repository layer coverage: 75%
- API endpoint coverage: 80%

11.1.3 Overall Unit Test Summary

Table 18: Overall Unit Test Summary

Service	Test Cases	Passed	Coverage
Java Backend (Auth)	24	24	85%
Python Backend (Business)	42	42	78%
Total	66	66	81%

All unit tests pass consistently, indicating that core business logic functions correctly in isolation. The test suite serves as a regression prevention mechanism.

11.2 Integration Testing Results

Integration tests verified repository operations, hashing, and FK integrity on MySQL for the Java service, and ORM mappings, transactions, and FK constraints on PostgreSQL for the Python service. All checks passed, confirming persistence correctness across both stacks.

11.3 Acceptance Testing Results

Acceptance tests covered end-to-end authentication scenarios (registration, login, logout), including invalid inputs and expired tokens.

Table 19: Acceptance Test Results

Feature	Scenarios	Passed	Status
User Registration	3	3	Pass
User Login	4	4	Pass
User Logout	2	2	Pass
Total	9	9	100%

The acceptance test suite covers authentication workflows comprehensively, including edge cases such as duplicate email registration, invalid email formats, weak passwords, and expired JWT tokens. The 100% pass rate indicates that the system correctly implements specified user requirements.

11.4 Stress Testing Results

JMeter stress tests evaluated performance under concurrent traffic.

Table 20: Stress Test Results Summary

Concurrent Users	Avg Response Time (ms)	Throughput (req/s)	Error Rate (%)
10	120	45	0
50	280	38	0
100	450	32	1.2

Key Findings: Handles 50 users under 300ms; at 100 users error rate stays below 2%; latency growth is linear after indexing hot queries (~40% gain).

11.5 CI/CD Pipeline Results

The GitHub Actions workflow runs on every push, executes all tests, builds images, and validates Docker Compose. Typical duration is 8–10 minutes (tests 4 minutes, builds 5 minutes) with 95% success, with rare retries for transient network issues.

11.6 Deployment Results

Docker Compose brings up all services with passing health checks; the stack is reachable at frontend (3000), Java API (8081), Python API (8000), MySQL (3307), and PostgreSQL (5433), confirming a reproducible local deployment.

12 Discussion

This section provides an analytical interpretation of the results obtained throughout the project, examining their alignment with the established objectives and reflecting on the technical, methodological, and organizational implications of the work conducted.

12.1 Achievement of Objectives

All project objectives were successfully met, demonstrating both the technical robustness of the solution and the methodological coherence of the development process. The adoption of a microservices architecture, implemented through independent Java and Python services, validated the project's intention to achieve modularity, scalability, and maintainability. These services were designed following well-structured user stories and user story mapping, which ensured a clear understanding of system requirements and prioritization logic.

Scrum practices played a central role in shaping the development workflow. Activities such as sprint planning, daily stand-ups, reviews, and retrospectives supported a coordinated and iterative construction of the platform. Likewise, the system analysis phase provided the necessary clarity to understand actors, components, and interactions, informing a software architecture that followed sound object-oriented principles and upheld a strict separation of concerns. Both backends achieved the required REST-based CRUD operations, and the React frontend demonstrated seamless interaction with both services.

Testing efforts contributed substantially to the reliability of the solution. A combination of unit, integration, acceptance, and stress testing validated system behavior from complementary perspectives. Furthermore, the GitHub Actions CI/CD pipeline automated the full testing and build lifecycle, enabling fast feedback and consistent evaluation of code quality. The entire platform was also containerized using Docker and Docker Compose, ensuring deployment reproducibility across environments. Comprehensive documentation and a final critical evaluation further confirmed the achievement of all defined objectives.

12.2 Architecture Effectiveness

The microservices architecture proved highly effective within the scope of this project. Its modular nature enabled independent development and deployment cycles, allowing each service to evolve at its own pace and with the most appropriate technology stack. This flexibility also strengthened fault isolation, preventing failures in one component from propagating across the system. Moreover, the architecture facilitated scaling decisions by enabling services to be replicated based on individual demand.

However, this design approach also introduced challenges. Service communication became more complex when compared to monolithic systems, requiring careful coordination of API contracts. Distributed behavior necessitated attention to synchronization, consistency, and potential transaction management considerations. Testing also increased in complexity, as validating distributed interactions requires more elaborate preparation. Despite these challenges, the architecture delivered clear advantages aligned with modern software engineering practices.

12.3 Testing Strategy Effectiveness

The testing strategy implemented throughout the project provided strong guarantees regarding software correctness and robustness. Unit testing proved especially useful during development, achieving an overall 81

12.4 CI/CD Pipeline Effectiveness

The CI/CD pipeline implemented through GitHub Actions demonstrated significant value by automating the testing and build lifecycle. By executing tests, running code quality checks, and building Docker images on every push and pull request, the pipeline enabled early detection of integration issues and ensured consistency across all development stages. The automated workflow minimized manual intervention and created an auditable record of every execution, contributing to transparency and reproducibility. Although the pipeline fulfilled its intended purpose, opportunities remain for future enhancement, such as incorporating automated deployments to a staging environment, integrating advanced code-quality tools like SonarQube, or including automated security scanning.

12.5 Containerization Benefits

Containerization played a strategic role in simplifying deployment and ensuring environmental consistency across the development lifecycle. By packaging each service with its dependencies inside a Docker container, the team avoided compatibility issues and streamlined continuous integration workflows. Docker Compose further facilitated orchestration by enabling simultaneous startup, configuration, and interconnection of all services. This setup eased scalability considerations and ensured service isolation, which in turn enhanced the system's maintainability and resilience. The ability to replicate the entire environment with a single command proved particularly valuable for testing and demonstrations.

12.6 Limitations and Their Impact

Certain constraints shaped the project's outcomes. Because payment processing was simulated rather than integrated with a real provider, the system could not evaluate scenarios requiring compliance with industry standards such as PCI DSS. Additionally, the absence of real-time monitoring limited the capacity to observe detailed performance metrics during execution, although this was acceptable for an academic project. Running the solution in a single deployment environment prevented the evaluation of geographically distributed workloads, and although the

12.7 Lessons Learned

Several important lessons emerged throughout the project. Early definition of API contracts proved essential for preventing integration issues and aligning expectations across services. The investment in testing from the outset significantly reduced debugging time and improved overall stability. Maintaining documentation continuously rather than at the end of the project facilitated communication, knowledge transfer, and onboarding. Containerization demonstrated clear benefits in simplifying environment management, while the adoption of Scrum practices ensured regular communication and prevented misalignment within the team. Together, these lessons reinforce the importance of methodical planning, continuous validation, and disciplined engineering practices in complex software projects.

13 Conclusions

This technical report presented the complete design, implementation, and evaluation of Eventify, a microservices-based event management and ticketing platform. The project demonstrated the practical application of contemporary software engineering practices, including microservices architecture, RESTful API development, containerization, automated testing, and continuous integration. Collectively, these components formed a coherent, scalable, and maintainable solution aligned with the project's objectives and with industry best practices.

13.1 Summary of Achievements

The project successfully met all of its planned objectives. The platform's architecture was designed and implemented following a microservices approach that enabled scalability, modularity, and independent service evolution. The development process was guided by comprehensive user stories and supported by Scrum practices, ensuring iterative delivery, clear prioritization, and continuous team alignment. System analysis and architectural design provided a robust foundation for implementation, resulting in two fully operational backend services—one in Java and one in Python—exposed through RESTful APIs and integrated seamlessly with a React-based frontend interface.

Testing efforts contributed significantly to the maturity of the solution. The project achieved

13.2 Key Contributions

From a software engineering perspective, the project offers several notable contributions. It demonstrates how microservices can be effectively applied in an event management scenario, highlighting the architectural benefits associated with service independence and technology diversity. The project also presents a comprehensive testing strategy that integrates unit, integration, acceptance, and stress testing, showcasing how different testing layers contribute to system reliability. In addition, the containerization of multiple services developed in different programming languages illustrates a reproducible deployment model that aligns with modern DevOps practices. The empirical evaluation performed through stress testing provides practical insights into performance behavior under load, while the extensive documentation traces the full software development lifecycle from requirements gathering to deployment.

13.3 Future Work Recommendations

Although the platform fulfills its intended objectives, several directions for future development have been identified. Expanding acceptance testing to cover all major user workflows would further strengthen reliability and ensure complete traceability between requirements and implemented features. Integrating a real payment gateway would enable the platform to support production-ready transactions, though this would require compliance with security standards and careful management of sensitive data. Introducing observability tools such as Prometheus and Grafana would improve real-time monitoring and facilitate performance diagnostics, while caching solutions like Redis could enhance responsiveness under heavy traffic.

Security remains an area with potential for further enhancement. Implementing additional measures such as rate limiting, automated security scanning within the CI/CD pipeline, and fraud detection mechanisms would reinforce the trustworthiness of the platform. Evaluating the system under geographically distributed deployments or integrating CDNs would offer insights into global performance. Finally, incorporating advanced features such as personalized event

recommendations, social media integration, or data-driven analytics dashboards would extend the platform's value and broaden its applicability.

13.4 Final Remarks

In conclusion, Eventify demonstrates that a microservices-based approach is well-suited for the design of modern event management platforms. The project's architecture, combined with rigorous testing, containerization, and automation, provides a robust and extensible foundation capable of supporting future enhancements and real-world deployment scenarios. Beyond delivering a functional system, the project offered valuable learning opportunities related to distributed architecture design, DevOps practices, agile methodologies, and full-stack integration. As such, it constitutes both a technically sound solution and a meaningful contribution to the practical understanding of contemporary software engineering.

References

References

- [1] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY: Manning Publications, 2018.
- [2] J. Smith and A. Johnson, "Digital Transformation in Event Management: A Survey," *Journal of Event Technology*, vol. 15, no. 3, pp. 45-62, 2023.
- [3] M. Cohn, *Succeeding with Agile: Software Development Using Scrum*. Boston, MA: Addison-Wesley Professional, 2009.
- [4] J. Turnbull, *The Docker Book: Containerization is the New Virtualization*. James Turnbull, 2014.
- [5] R. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.
- [6] M. Fowler, "Microservices," *martinfowler.com*, 2015. [Online]. Available: <https://martinfowler.com/articles/microservices.html>
- [7] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA: O'Reilly Media, 2015.
- [8] Spring Framework Documentation, "Spring Boot Reference Documentation," 2024. [Online]. Available: <https://spring.io/projects/spring-boot>
- [9] FastAPI Documentation, "FastAPI - Modern, Fast, Web Framework for Building APIs," 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [10] React Documentation, "React - A JavaScript Library for Building User Interfaces," 2024. [Online]. Available: <https://react.dev/>

Acknowledgements

The authors thank Eng. Carlos Andrés Sierra, M.Sc., Lecturer at Universidad Nacional de Colombia, for his guidance, supervision, and valuable feedback throughout this project. His expertise in software engineering and commitment to academic excellence greatly contributed to the success of this work.

We also acknowledge the Software Engineering Seminar course at Universidad Distrital Francisco José de Caldas for providing the framework, resources, and academic environment that made this project possible.

Special thanks to the open-source community for providing excellent tools and frameworks including Spring Boot, FastAPI, React, Docker, and all other technologies that enabled this project.

Glossary

Additional terms:

Microservices Architecture An architectural pattern that structures an application as a collection of loosely coupled, independently deployable services.

RESTful API An API that follows REST (Representational State Transfer) principles, using standard HTTP methods and stateless communication.

Containerization The process of packaging an application with its dependencies into a container for consistent deployment across environments.

CI/CD Continuous Integration and Continuous Deployment - practices that automate testing and deployment processes.

Test Coverage A measure of the percentage of code executed by tests, indicating how thoroughly code is tested.

Acceptance Testing Testing that validates whether a system meets acceptance criteria and user requirements.

Stress Testing Performance testing that evaluates system behavior under extreme load conditions.

ORM Object-Relational Mapping - a technique for converting data between incompatible type systems in object-oriented programming languages.

SPA Single-Page Application - a web application that loads a single HTML page and dynamically updates content.

CORS Cross-Origin Resource Sharing - a mechanism that allows web pages to request resources from a different domain.